WYDZIAŁ FIZYKI
i INFORMATYKI STOSOWANEJ
Uniwersytet Łódzki

Patryk Michalak

Kierunek: informatyka
Specjalność: informatyka stosowana
Specjalizacja: Algorytmy i Programowanie
Numer albumu: 410205

Niskopoziomowa emulacja sprzętowa implementacja architekturę systemową konsoli Game Boy

Praca inżynierska
wykonana pod kierunkiem
prof. dr. hab. Pawła Kowalczyka
w Katedrze Fizyki Ciała Stałego
WFiIS UŁ

Łódź 2026

Spis treści

1 Wstęp **4**
1.1 Opis problemu . . . 4
1.2 Emulatory . . . 4
1.3 Założenia i cel pracy . . . 4
1.4 Grupa docelowa . . . 5
1.5 Inne emulatory Game Boya . . . 5

2 Analiza architektury systemowej konsoli Game Boy **6**
2.1 Specyfikacja techniczna . . . 6
2.2 Procesor i SoC . . . 6
2.2.1 Model czasowy systemu . . . 6
2.3 Zestaw Instrukcji . . . 7
2.3.1 Instrukcje transferu danych . . . 7
2.3.2 Instrukcje arytmetyczne . . . 7
2.3.3 Instrukcje logiczne i bitowe . . . 7
2.3.4 Instrukcje przesunięć i rotacji . . . 10
2.3.5 Instrukcje sterowania przepływem programu . . . 10
2.4 Mapowanie pamięci w konsoli Game Boy . . . 10
2.4.1 Ogólny układ przestrzeni adresowej . . . 10
2.4.2 Pamięć programu i bankowanie ROM . . . 11
2.4.3 Pamięć wideo . . . 11
2.4.4 Pamięć robocza i obszar echo . . . 11
2.4.5 Mapowanie urządzeń wejścia/wyjścia . . . 11
2.4.6 Pamięć wysokiej prędkości . . . 11
2.5 System przerwań w konsoli Game Boy . . . 11
2.5.1 Źródła przerwań . . . 11
2.5.2 Rejestry sterujące systemem przerwań . . . 12
2.5.3 Mechanizm obsługi przerwania . . . 12
2.5.4 Priorytety przerwań . . . 12
2.5.5 Rola systemu przerwań w architekturze konsoli . . . 12
2.6 Płyta główna . . . 13
2.6.1 Jednostka centralna . . . 13
2.6.2 Pamięć operacyjna i pamięć wideo . . . 13
2.6.3 Układ graficzny . . . 13
2.6.4 Układ dźwiękowy . . . 14
2.6.5 Interfejs kartridża . . . 14
2.6.6 Układy wejścia i wyjścia . . . 14
2.7 Kartridże systemu Game Boy . . . 14
2.7.1 Kartridże bez kontrolera banków pamięci (ROM Only) . . . 14
2.7.2 Kartridże z kontrolerem MBC1 . . . 14
2.7.3 Kartridże z kontrolerem MBC2 . . . 15
2.7.4 Kartridże z kontrolerem MBC3 . . . 15
2.7.5 Kartriże z kontrolerem MBC4 . . . 15
2.7.6 Kartridże z kontrolerem MBC5 . . . 15
2.7.7 Kartridże specjalizowane . . . 15
2.7.8 Struktura fizyczna kartridża . . . 15

3 Model działania systemu Game Boy **16**
3.1 Inicjalizacja systemu . . . 16
3.1.1 Stan początkowy procesora . . . 16
3.1.2 Konfiguracja pamięci i przestrzeni adresowej . . . 16
3.1.3 Inicjalizacja układu graficznego . . . 16
3.1.4 Przygotowanie systemu przerwań . . . 16
3.1.5 Przekazanie sterowania programowi użytkownika . . . 16
3.1.6 Znaczenie procesu inicjalizacji . . . 16
3.2 Cykl pracy procesora . . . 17
3.2.1 Model cyklu rozkazowego . . . 17
3.2.2 Synchronizacja z zegarem systemowym . . . 17
3.2.3 Dostęp do pamięci i operacje magistrali . . . 17

3.2.4 Obsługa przerwań . . . 17
3.2.5 Znaczenie cyklu pracy dla działania systemu . . . 17
3.3 Magistrala pamięciowa i charakterystyka działania pamięci . . . 18
3.3.1 Organizacja magistrali pamięciowej . . . 18
3.3.2 Mapowanie przestrzeni adresowej . . . 18
3.3.3 Rodzaje pamięci w systemie . . . 18
3.3.4 Charakterystyka operacji pamięciowych . . . 18
3.3.5 Rola systemu pamięci w działaniu konsoli . . . 18
3.4 Generowanie klatki obrazu . . . 19
3.4.1 Format zapisu grafiki w pamięci VRAM . . . 19
3.4.2 Renderowanie tła . . . 19
3.4.3 Pamięć OAM i reprezentacja obiektów . . . 19
3.4.4 Renderowanie obiektów . . . 19
3.4.5 DMA i bezpośredni transfer danych do OAM . . . 20
3.5 System przerwań . . . 20
3.5.1 Warunki występowania przerwań . . . 20
3.5.2 Rodzaje przerwań . . . 20
3.5.3 Przebieg obsługi przerwania . . . 20
3.5.4 Funkcje realizowane przez przerwania . . . 21
3.5.5 Znaczenie mechanizmu przerwań . . . 21

4 Implementacja **22**
4.1 Wykorzystane narzędzia i zasoby . . . 22
4.1.1 Funkcje biblioteki Raylib . . . 22
4.1.2 Struktury biblioteki Raylib . . . 22
4.1.3 Funkcje biblioteki fmt . . . 23
4.2 Implementacja modułu obsługi kartridża . . . 23
4.2.1 Struktura nagłówka ROM . . . 23
4.2.2 Proces ładowania obrazu ROM . . . 23
4.2.3 Dostęp do danych kartridża . . . 24
4.2.4 Weryfikacja poprawności nagłówka . . . 24
4.2.5 Ograniczenia implementacji . . . 24
4.3 Implementacja magistrali systemowej . . . 24
4.3.1 Operacje odczytu z magistrali . . . 24
4.3.2 Operacje zapisu do magistrali . . . 24
4.3.3 Operacje na słowach 16-bitowych . . . 25
4.3.4 Znaczenie architektoniczne magistrali . . . 25
4.4 Implementacja pamięci operacyjnej RAM . . . 25
4.5 Implementacja stosu procesora . . . 26
4.6 Implementacja systemowego licznika czasowego . . . 26
4.7 Implementacja procesora . . . 26
4.7.1 Reprezentacja i przechowywanie instrukcji . . . 26
4.7.2 Struktura procesora i organizacja cyklu wykonawczego . . . 27
4.7.3 Organizacja cyklu wykonawczego . . . 29
4.7.4 Warstwa dostępu do rejestrów . . . 30
4.7.5 Wykonanie w środowisku wielowątkowym . . . 30
4.8 Implementacja przetwarzania grafiki . . . 30
4.8.1 Reprezentacja rejestrów LCD i obsługa palet . . . 30
4.8.2 Renderowanie tła oraz obiektów OAM . . . 32
4.8.3 Maszyna stanów kontrolera LCD . . . 33
4.9 Implementacja interfejsu graficznego emulatora . . . 33
4.9.1 Opis klasy . . . 33
4.9.2 Interfejs publiczny . . . 34
4.9.3 Przykład działania korzystania z klasy RayboyUI . . . 34
4.9.4 Kluczowe założenia projektowe . . . 34
4.9.5 Struktura klasy . . . 35
4.9.6 Główna pętla interfejsu . . . 35
4.9.7 Inicjalizacja interfejsu . . . 35
4.9.8 Aktualizacja obrazu ekranu . . . 36
4.9.9 Renderowanie obrazu emulatora . . . 37
4.9.10 Obsługa wejścia użytkownika . . . 37

4.9.11 Podgląd kafelków graficznych . . . 38
4.9.12 Panel statystyk . . . 38
4.9.13 Ekran Pomocy . . . 39
4.9.14 Podsumowanie . . . 39
4.10 Weryfikacja implementacji emulatora . . . 40
4.10.1 Metodyka weryfikacji . . . 40
4.10.2 Narzędzia testowe . . . 40
4.10.3 Testy porównawcze . . . 40
4.11 Problemy implementacyjne . . . 41

5 Działanie programu **42**
5.1 Struktura Programu . . . 42
5.2 Sterowanie w trakcji działania emulatora . . . 42

6 Zakończenie **43**

1 Wstęp

1.1 Opis problemu

Konsola *Game Boy*, wyprodukowana przez firmę *Nintendo* i wprowadzona na rynek japoński 21 kwietnia 1989 roku, stała się jednym z najbardziej rozpoznawalnych urządzeń w historii elektronicznej rozrywki. Game Boy odniosła znaczący sukces komercyjny – w ciągu pierwszych lat od premiery sprzedano ponad 20 milionów egzemplarzy. W kolejnych latach konsola została wprowadzona na rynki zachodnie, gdzie również zdobyła dużą popularność. Łączna sprzedaż wszystkich wariantów urządzenia uczyniła ją jedną z najlepiej sprzedających się konsol przenośnych w historii.

Gry dla konsoli były dystrybuowane w postaci wymiennych kartridży (Game Pak), zawierających pamięć ROM z kodem programu, danymi graficznymi oraz dźwiękowymi. W niektórych przypadkach kartridże wyposażone były dodatkowo w pamięć RAM oraz baterię podtrzymującą stan gry. Architektura sprzętowa konsoli opierała się na 8-bitowym procesorze taktowanym częstotliwością około 4,19 MHz, a obraz generowany był w rozdzielczości 160×144 pikseli przy czterech poziomach szarości.

Wraz z zakończeniem produkcji konsoli przez *Nintendo* dostęp do oryginalnego sprzętu stał się ograniczony. Znaczna część tytułów wydanych wyłącznie na tę platformę nie została oficjalnie przeniesiona na nowsze systemy. W konsekwencji wiele gier pozostaje dostępnych jedynie poprzez zachowane egzemplarze konsoli oraz kartridże, co utrudnia ich użytkowanie oraz archiwizację.

1.2 Emulatory

Jednym z rozwiązań problemu ograniczonej dostępności oryginalnego sprzętu są emulatory, czyli programy komputerowe umożliwiające odtworzenie funkcjonalności danego systemu sprzętowego w środowisku wirtualnym. Emulator odwzorowuje działanie procesora, pamięci, układów graficznych oraz urządzeń wejścia/wyjścia w sposób pozwalający na uruchamianie oryginalnego oprogramowania.

Emulatory znajdują zastosowanie nie tylko w środowisku graczy, lecz również w badaniach nad architekturą systemów komputerowych, analizie kompatybilności oprogramowania oraz cyfrowej archiwizacji dziedzictwa technologicznego. Umożliwiają one zachowanie historycznej wartości gier i systemów, które w przeciwnym razie mogłyby zostać utracone wraz z degradacją fizycznych nośników.

Wyróżnia się dwa podstawowe podejścia do emulacji:

- **Emulacja pełna** – polegająca na wiernym odwzorowaniu wszystkich kluczowych komponentów sprzętowych systemu, w tym procesora, pamięci, grafiki oraz dźwięku.
- **Emulacja częściowa** – obejmująca jedynie wybrane elementy systemu, co może ograniczać kompatybilność z oryginalnym oprogramowaniem.

W kontekście niniejszej pracy przyjęto założenie realizacji emulacji pełnej w zakresie podstawowej funkcjonalności konsoli.

1.3 Założenia i cel pracy

Celem niniejszej pracy jest zaprojektowanie oraz implementacja emulatora konsoli Game Boy w języku programowania C++, z wykorzystaniem biblioteki Raylib do obsługi warstwy graficznej i wejścia użytkownika. Projekt zakłada odwzorowanie podstawowych mechanizmów działania oryginalnej architektury sprzętowej w stopniu umożliwiającym uruchamianie wybranej klasy gier.

Zakres pracy obejmuje:

- implementację emulacji procesora oraz mapy pamięci konsoli,
- odwzorowanie mechanizmu wyświetlania obrazu,
- obsługę wejścia użytkownika z wykorzystaniem klawiatury,
- możliwość wczytywania obrazów ROM typu *ROM Only*,
- zapewnienie wieloplatformowości (Windows, Linux, macOS).

W ramach projektu przyjęto ograniczenie do obsługi podstawowego typu kartridża zawierającego wyłącznie pamięć ROM, bez dodatkowych kontrolerów pamięci (MBC). Emulator przeznaczony jest do uruchamiania cyfrowych kopii gier pozyskanych przez użytkownika z posiadanych nośników.

Poprawność działania systemu zostanie zweryfikowana poprzez porównanie wyników emulacji z publicznie dostępnymi testami zgodności opracowanymi przez społeczność dokumentującą architekturę konsoli.

1.4 Grupa docelowa

Odbiorcami opracowanego systemu są użytkownicy posiadający legalne kopie gier przeznaczonych na konsolę Game Boy, zainteresowani ich uruchamianiem na współczesnych komputerach osobistych. Emulator może stanowić również narzędzie edukacyjne dla osób zainteresowanych architekturą systemów wbudowanych oraz mechanizmami niskopoziomowego działania sprzętu.

1.5 Inne emulatory Game Boya

Na przestrzeni lat powstało wiele emulatorów Game Boya o różnym stopniu dokładności, przenośności i przeznaczeniu. Poniżej opisano najważniejsze projekty open source, które stanowią punkt odniesienia dla współczesnych implementacji.

SameBoy `SameBoy` [6] jest jednym z najbardziej dokładnych dostępnych emulatorów Game Boya i Game Boya Color. Projekt jest wieloplatformowy – obsługuje systemy macOS, Windows oraz platformy uniksopodobne. Oprócz wysokiej wierności emulacji sprzętu, `SameBoy` oferuje rozbudowane narzędzia diagnostyczne oraz emulację akcesoriów, takich jak Game Boy Camera i Game Boy Printer.

mGBA `mGBA` [10] to wieloplatformowy emulator Game Boya Advance, obsługujący również Game Boya i Game Boya Color. Projekt kładzie nacisk zarówno na szybkość działania, jak i na dokładność emulacji. Emulator dostępny jest na systemy Windows, macOS, Linux, a także jako homebrew na konsole Nintendo Switch, Wii oraz PlayStation Vita. `mGBA` wspiera skryptowanie w języku Lua, co czyni go popularnym narzędziem wśród twórców narzędzi pomocniczych i speedrunnerów.

Gearboy `Gearboy` [11] to wieloplatformowy emulator Game Boya i Game Boya Color napisany w C++. Projekt wyróżnia się dokładną emulacją procesora i kontrolera LCD, przechodzi testy `cpu_instrs.gb` oraz `instr_timing.gb` autorstwa Blarrga. Emulator posiada rozbudowany debugger z deasemblerem, punktami przerwań oraz inspektorem pamięci VRAM.

VisualBoyAdvance-M `VisualBoyAdvance-M` [15] jest następcą oryginalnego emulatora `VisualBoyAdvance` i obsługuje Game Boya, Game Boya Color oraz Game Boya Advance. Projekt dostępny jest na systemy Windows, macOS, Linux oraz Android. Emulator wspiera popularne systemy kodów cheat, takie jak GameShark i CodeBreaker.

2 Analiza architektury systemowej konsoli Game Boy

2.1 Specyfikacja techniczna

Zespół *Nintendo Research And Development 1*, pod przewodnictwem Gunpei Yokoi i Satoru Okady, zaprojektował 8-bitową konsolę Game Boy. Charakteryzuje się wyświetlaczem matrycowym o rozdzielczości 160x144 pikseli, D-padem, czterema przyciskami gry i jednym głośnikiem. Używa stworzonych na potrzebe konsoli kartridże *GamePak*. Zasilana była przez 4 baterie AA. Gracze mogli korzystać z kabla Game Link Cable do połączenia dwóch Game Boyów dla rozgrywki wieloosobowej bądź transferu danych dla wspierających gier. Chociaż wyświetlacz monochromatyczny był gorszy niż u konkurencji, utrzymanie konsoli w jednym kolorze przyczyniło się do większonej żywotności i wymaganej liczbie baterii. [2].

2.2 Procesor i SoC

Procesorem Gameboya był specjalnie wytworzyny na potrzeby konsoli 8-bitowy procesor SM83 firmy Sharp. [4]. Procesor był wzorowany na dwóch innych procesorach- 8080 firmy Intel i Z80 firmy Zilog. Procesor znajdował się na płytce wraz innymi komponentami takimi jak pamięć RAM i ROM. Cała płytka jest określana jako SoC (System on Chip) i oryginalny Game Boy zawierał DMG-CPU znany także jako Sharp LR35902 [1]. SoC był wpinany do płyty głównej, która zawierała dodatkową pamięć RAM i Video RAM (VRAM) . Płyta główna łączyła także inne kompomenty jak ekran, głośnik czy kontroler.

Procesor zawiera osiem 8-bitowych rejestrów: A,B,C,D,E,F,H,L. Rejestry mogą łączyć się w 16-bitowe rejestry: AF, BC, DE, HL. Rejestr F jest używany jako flagi procesora w wypadku operacji arymetycznych:

- Bit 7 - Zero (Z), jeśli operacja zwróciła zero,
- Bit 6 - Negacja (N), używana gdy ostatnia operacją była porównaniem bądź odejmowaniem,
- Bit 5 - Half Carry (H), jeśli doszło do przesunięcia na bicie 3,
- Bit 4 - Carry (C), jeśli doszło do przesunięcia na bicie 7.

Bity od 3 do 0 są nieużywane i zawsze powinny być zerowane. Rejestr A jest używany jako akumulator w operacja arytmetcznych.

2.2.1 Model czasowy systemu

Działanie konsoli Game Boy oparte jest na synchronicznym modelu czasowym, w którym wszystkie główne komponenty systemu — procesor, układ graficzny oraz timery sprzętowe — pracują w oparciu o wspólne źródło taktowania. Nominalna częstotliwość zegara systemowego wynosi 4,194304 MHz, co determinuje tempo wykonywania instrukcji procesora oraz przebieg cykli pracy pozostałych bloków funkcjonalnych.

Cykle maszynowe procesora Procesor wykonuje instrukcje w jednostkach zwanych cyklami maszynowymi (ang. *machine cycles*), z których każdy obejmuje określoną liczbę taktów zegara systemowego. Czas wykonania instrukcji zależy od jej złożoności i wynosi zazwyczaj od jednego do kilku cykli maszynowych. Model czasowy procesora ma charakter deterministyczny, co oznacza, że liczba cykli wymaganych do wykonania danej instrukcji jest stała i niezależna od kontekstu wykonania.

Synchronizacja z układem graficznym Układ graficzny pracuje równolegle z procesorem i cyklicznie przechodzi przez zdefiniowane fazy generowania obrazu. Każda ramka obrazu składa się z sekwencji linii skanowania, a przebieg ich przetwarzania wyznacza rytm pracy całego systemu. W określonych fazach generowania obrazu dostęp procesora do wybranych obszarów pamięci wideo jest ograniczony, co stanowi bezpośrednią konsekwencję współdzielenia zasobów pamięci pomiędzy jednostkami systemu.

System timerów sprzętowych Konsola wyposażona jest w sprzętowe układy odmierzania czasu, których działanie jest bezpośrednio powiązane z zegarem systemowym. Timery te generują

zdarzenia okresowe wykorzystywane do synchronizacji logiki programu, pomiaru czasu oraz inicjowania przerwań sprzętowych. Ich konfiguracja pozwala na wybór różnych częstotliwości zliczania poprzez dzielenie sygnału zegarowego.

2.3 Zestaw Instrukcji

Zestaw Rozkazów (instruction set architecture, ISA) (rysunek 1 i rysunek 2) stanowi rozwinięcie koncepcji znanych z mikroprocesorów rodziny Z80, z licznymi uproszczeniami i modyfikacjami dostosowanymi do zastosowań w systemie przenośnym. Wszytkie rozkazy można systematycznie sklasyfikować na poszczególne grupy.

Każda instrukcja zapisywana jest w formie kodu binarnego (opcode) od rozmiarze 1 bajtów (2 w przypadku gdy pierwszy bajt to 0xCB). Tabela pokazująca tworzenie kodów binarnych dla części instrukcji pokazana jest na rysunku 1, gdzie na rysunku 2 są przedstowione wszytkie instrukcje z przedrostkiem CB. W szczególności aby utworzyć kod dla OR A, H od opcodzie równym 0x starsza, 4 bitowa cześć kodu binarnego wybierana jest z odpowiedniego wiersza i w omawianym tu przypadku wynosi B, gdzie młodsze bity kodują kolumnę 4. Wybrana instrukcja jest zaznaczona na rysunku 1

Instrukcje tej grupy odpowiadają za przemieszczanie informacji pomiędzy rejestrami, pamięcią oraz natychmiastowymi wartościami liczbowymi. Do najważniejszych operacji należą:

2.3.1 Instrukcje transferu danych

- **LD (Load)** – realizuje kopiowanie danych pomiędzy rejestrami ogólnego przeznaczenia (A, B, C, D, E, H, L), parami rejestrów oraz komórkami pamięci adresowanymi bezpośrednio lub pośrednio.
- **LDH** – specjalizowana forma transferu do i z obszaru pamięci o wysokim adresie (High RAM).
- **PUSH / POP** – zapis i odczyt danych na stosie, z wykorzystaniem wskaźnika stosu SP.

Instrukcje transferu nie modyfikują danych źródłowych (z wyjątkiem operacji wykonywanych na stosie), a ich podstawową funkcją jest reorganizacja informacji w przestrzeni rejestrowo-pamięciowej procesora.

2.3.2 Instrukcje arytmetyczne

Instrukcje arytmetyczne realizują operacje matematyczne na liczbach całkowitych. W SM83 operacje te wykonywane są głównie na akumulatorze A. Do kluczowych operacji należą:

- **ADD / ADC** – dodawanie bez i z uwzględnieniem flagi przeniesienia,
- **SUB / SBC** – odejmowanie bez i z uwzględnieniem przeniesienia,
- **INC / DEC** – inkrementacja i dekrementacja rejestrów oraz komórek pamięci,
- **DAA** – korekcja wyniku dodawania do postaci BCD,
- **CP** – porównanie wartości bez zapisu wyniku (modyfikowane są jedynie flagi).

Instrukcje te aktualizują rejestr flag (Z, N, H, C), który odzwierciedla właściwości wyniku operacji.

2.3.3 Instrukcje logiczne i bitowe

Grupa ta obejmuje operacje algebry Boole'a oraz manipulacje pojedynczymi bitami. Do najważniejszych należą:

- **AND, OR, XOR** – operacje logiczne na akumulatorze,
- **CPL** – negacja bitowa zawartości akumulatora,
- **BIT** – test wybranego bitu,
- **SET / RES** – ustawienie lub wyzerowanie wskazanego bitu.

Instrukcje bitowe umożliwiają precyzyjną kontrolę struktur danych, co jest szczególnie istotne w systemach o ograniczonych zasobach pamięci.

Game Boy CPU (SM83) instruction set (JSON)

	x0	x1	x2	x3	x4	x5	x6	x7	x8	x9	xA	xB	xC	xD	xE	xF
0x	NOP 1 4 - - - -	LD BC, n16 3 12 - - - -	LD [BC], A 1 8 - - - -	INC BC 1 8 - - - -	INC B 1 4 Z 0 H -	DEC B 1 4 Z 1 H -	LD B, n8 2 8 - - - -	RLCA 1 4 0 0 0 C	LD [a16], SP 3 20 - - - -	ADD HL, BC 1 8 - 0 H C	LD A, [BC] 1 8 - - - -	DEC BC 1 8 - - - -	INC C 1 4 Z 0 H -	DEC C 1 4 Z 1 H -	LD C, n8 2 8 - - - -	RRCA 1 4 0 0 0 C
1x	STOP n8 2 4 - - - -	LD DE, n16 3 12 - - - -	LD [DE], A 1 8 - - - -	INC DE 1 8 - - - -	INC D 1 4 Z 0 H -	DEC D 1 4 Z 1 H -	LD D, n8 2 8 - - - -	RLA 1 4 0 0 0 C	JR e8 2 12 - - - -	ADD HL, DE 1 8 - 0 H C	LD A, [DE] 1 8 - - - -	DEC DE 1 8 - - - -	INC E 1 4 Z 0 H -	DEC E 1 4 Z 1 H -	LD E, n8 2 8 - - - -	RRA 1 4 0 0 0 C
2x	JR NZ, e8 2 12/8 - - - -	LD HL, n16 3 12 - - - -	LD [HL+], A 1 8 - - - -	INC HL 1 8 - - - -	INC H 1 4 Z 0 H -	DEC H 1 4 Z 1 H -	LD H, n8 2 8 - - - -	DAA 1 4 Z - 0 C	JR Z, e8 2 12/8 - - - -	ADD HL, HL 1 8 - 0 H C	LD A, [HL+] 1 8 - - - -	DEC HL 1 8 - - - -	INC L 1 4 Z 0 H -	DEC L 1 4 Z 1 H -	LD L, n8 2 8 - - - -	CPL 1 4 - 1 1 -
3x	JR NC, e8 2 12/8 - - - -	LD SP, n16 3 12 - - - -	LD [HL-], A 1 8 - - - -	INC SP 1 8 - - - -	INC [HL] 1 12 Z 0 H -	DEC [HL] 1 12 Z 1 H -	LD [HL], n8 2 12 - - - -	SCF 1 4 - 0 0 1	JR C, e8 2 12/8 - - - -	ADD HL, SP 1 8 - 0 H C	LD A, [HL-] 1 8 - - - -	DEC SP 1 8 - - - -	INC A 1 4 Z 0 H -	DEC A 1 4 Z 1 H -	LD A, n8 2 8 - - - -	CCF 1 4 - 0 0 C
4x	LD B, B 1 4 - - - -	LD B, C 1 4 - - - -	LD B, D 1 4 - - - -	LD B, E 1 4 - - - -	LD B, H 1 4 - - - -	LD B, L 1 4 - - - -	LD B, [HL] 1 8 - - - -	LD B, A 1 4 - - - -	LD C, B 1 4 - - - -	LD C, C 1 4 - - - -	LD C, D 1 4 - - - -	LD C, E 1 4 - - - -	LD C, H 1 4 - - - -	LD C, L 1 4 - - - -	LD C, [HL] 1 8 - - - -	LD C, A 1 4 - - - -
5x	LD D, B 1 4 - - - -	LD D, C 1 4 - - - -	LD D, D 1 4 - - - -	LD D, E 1 4 - - - -	LD D, H 1 4 - - - -	LD D, L 1 4 - - - -	LD D, [HL] 1 8 - - - -	LD D, A 1 4 - - - -	LD E, B 1 4 - - - -	LD E, C 1 4 - - - -	LD E, D 1 4 - - - -	LD E, E 1 4 - - - -	LD E, H 1 4 - - - -	LD E, L 1 4 - - - -	LD E, [HL] 1 8 - - - -	LD E, A 1 4 - - - -
6x	LD H, B 1 4 - - - -	LD H, C 1 4 - - - -	LD H, D 1 4 - - - -	LD H, E 1 4 - - - -	LD H, H 1 4 - - - -	LD H, L 1 4 - - - -	LD H, [HL] 1 8 - - - -	LD H, A 1 4 - - - -	LD L, B 1 4 - - - -	LD L, C 1 4 - - - -	LD L, D 1 4 - - - -	LD L, E 1 4 - - - -	LD L, H 1 4 - - - -	LD L, L 1 4 - - - -	LD L, [HL] 1 8 - - - -	LD L, A 1 4 - - - -
7x	LD [HL], B 1 8 - - - -	LD [HL], C 1 8 - - - -	LD [HL], D 1 8 - - - -	LD [HL], E 1 8 - - - -	LD [HL], H 1 8 - - - -	LD [HL], L 1 8 - - - -	HALT 1 4 - - - -	LD [HL], A 1 8 - - - -	LD A, B 1 4 - - - -	LD A, C 1 4 - - - -	LD A, D 1 4 - - - -	LD A, E 1 4 - - - -	LD A, H 1 4 - - - -	LD A, L 1 4 - - - -	LD A, [HL] 1 8 - - - -	LD A, A 1 4 - - - -
8x	ADD A, B 1 4 Z 0 H C	ADD A, C 1 4 Z 0 H C	ADD A, D 1 4 Z 0 H C	ADD A, E 1 4 Z 0 H C	ADD A, H 1 4 Z 0 H C	ADD A, L 1 4 Z 0 H C	ADD A, [HL] 1 8 Z 0 H C	ADD A, A 1 4 Z 0 H C	ADC A, B 1 4 Z 0 H C	ADC A, C 1 4 Z 0 H C	ADC A, D 1 4 Z 0 H C	ADC A, E 1 4 Z 0 H C	ADC A, H 1 4 Z 0 H C	ADC A, L 1 4 Z 0 H C	ADC A, [HL] 1 8 Z 0 H C	ADC A, A 1 4 Z 0 H C
9x	SUB A, B 1 4 Z 1 H C	SUB A, C 1 4 Z 1 H C	SUB A, D 1 4 Z 1 H C	SUB A, E 1 4 Z 1 H C	SUB A, H 1 4 Z 1 H C	SUB A, L 1 4 Z 1 H C	SUB A, [HL] 1 8 Z 1 H C	SUB A, A 1 4 1 1 0 0	SBC A, B 1 4 Z 1 H C	SBC A, C 1 4 Z 1 H C	SBC A, D 1 4 Z 1 H C	SBC A, E 1 4 Z 1 H C	SBC A, H 1 4 Z 1 H C	SBC A, L 1 4 Z 1 H C	SBC A, [HL] 1 8 Z 1 H C	SBC A, A 1 4 Z 1 H -
Ax	AND A, B 1 4 Z 0 1 0	AND A, C 1 4 Z 0 1 0	AND A, D 1 4 Z 0 1 0	AND A, E 1 4 Z 0 1 0	AND A, H 1 4 Z 0 1 0	AND A, L 1 4 Z 0 1 0	AND A, [HL] 1 8 Z 0 1 0	AND A, A 1 4 Z 0 1 0	XOR A, B 1 4 Z 0 0 0	XOR A, C 1 4 Z 0 0 0	XOR A, D 1 4 Z 0 0 0	XOR A, E 1 4 Z 0 0 0	XOR A, H 1 4 Z 0 0 0	XOR A, L 1 4 Z 0 0 0	XOR A, [HL] 1 8 Z 0 0 0	XOR A, A 1 4 1 0 0 0
Bx	OR A, B 1 4 Z 0 0 0	OR A, C 1 4 Z 0 0 0	OR A, D 1 4 Z 0 0 0	OR A, E 1 4 Z 0 0 0	OR A, H 1 4 Z 0 0 0	OR A, L 1 4 Z 0 0 0	OR A, [HL] 1 8 Z 0 0 0	OR A, A 1 4 Z 0 0 0	CP A, B 1 4 Z 1 H C	CP A, C 1 4 Z 1 H C	CP A, D 1 4 Z 1 H C	CP A, E 1 4 Z 1 H C	CP A, H 1 4 Z 1 H C	CP A, L 1 4 Z 1 H C	CP A, [HL] 1 8 Z 1 H C	CP A, A 1 4 1 1 0 0
Cx	RET NZ 1 20/8 - - - -	POP BC 1 12 - - - -	JP NZ, a16 3 16/12 - - - -	JP a16 3 16 - - - -	CALL NZ, a16 3 24/12 - - - -	PUSH BC 1 16 - - - -	ADD A, n8 2 8 Z 0 H C	RST $00 1 16 - - - -	RET Z 1 20/8 - - - -	RET 1 16 - - - -	JP Z, a16 3 16/12 - - - -	PREFIX 1 4 - - - -	CALL Z, a16 3 24/12 - - - -	CALL a16 3 24 - - - -	ADC A, n8 2 8 Z 0 H C	RST $08 1 16 - - - -
Dx	RET NC 1 20/8 - - - -	POP DE 1 12 - - - -	JP NC, a16 3 16/12 - - - -	—	CALL NC, a16 3 24/12 - - - -	PUSH DE 1 16 - - - -	SUB A, n8 2 8 Z 1 H C	RST $10 1 16 - - - -	RET C 1 20/8 - - - -	RETI 1 16 - - - -	JP C, a16 3 16/12 - - - -	—	CALL C, a16 3 24/12 - - - -	—	SBC A, n8 2 8 Z 1 H C	RST $18 1 16 - - - -
Ex	LDH [a8], A 2 12 - - - -	POP HL 1 12 - - - -	LDH [C], A 1 8 - - - -	—	—	PUSH HL 1 16 - - - -	AND A, n8 2 8 Z 0 1 0	RST $20 1 16 - - - -	ADD SP, e8 2 16 0 0 H C	JP HL 1 4 - - - -	LD [a16], A 3 16 - - - -	—	—	—	XOR A, n8 2 8 Z 0 0 0	RST $28 1 16 - - - -
Fx	LDH A, [a8] 2 12 - - - -	POP AF 1 12 Z N H C	LDH A, [C] 1 8 - - - -	DI 1 4 - - - -	—	PUSH AF 1 16 - - - -	OR A, n8 2 8 Z 0 0 0	RST $30 1 16 - - - -	LD HL, SP + e8 2 12 0 0 H C	LD SP, HL 1 8 - - - -	LD A, [a16] 3 16 - - - -	EI 1 4 - - - -	—	—	CP A, n8 2 8 Z 1 H C	RST $38 1 16 - - - -

Rysunek 1: Tablica rozmieszczenia instrukcji procesora SM83. Źródło: [3]

🔗 Prefixed ($CB $xx)

	x0	x1	x2	x3	x4	x5	x6	x7	x8	x9	xA	xB	xC	xD	xE	xF
0x	RLC B 2 8 Z 0 0 C	RLC C 2 8 Z 0 0 C	RLC D 2 8 Z 0 0 C	RLC E 2 8 Z 0 0 C	RLC H 2 8 Z 0 0 C	RLC L 2 8 Z 0 0 C	RLC [HL] 2 16 Z 0 0 C	RLC A 2 8 Z 0 0 C	RRC B 2 8 Z 0 0 C	RRC C 2 8 Z 0 0 C	RRC D 2 8 Z 0 0 C	RRC E 2 8 Z 0 0 C	RRC H 2 8 Z 0 0 C	RRC L 2 8 Z 0 0 C	RRC [HL] 2 16 Z 0 0 C	RRC A 2 8 Z 0 0 C
1x	RL B 2 8 Z 0 0 C	RL C 2 8 Z 0 0 C	RL D 2 8 Z 0 0 C	RL E 2 8 Z 0 0 C	RL H 2 8 Z 0 0 C	RL L 2 8 Z 0 0 C	RL [HL] 2 16 Z 0 0 C	RL A 2 8 Z 0 0 C	RR B 2 8 Z 0 0 C	RR C 2 8 Z 0 0 C	RR D 2 8 Z 0 0 C	RR E 2 8 Z 0 0 C	RR H 2 8 Z 0 0 C	RR L 2 8 Z 0 0 C	RR [HL] 2 16 Z 0 0 C	RR A 2 8 Z 0 0 C
2x	SLA B 2 8 Z 0 0 C	SLA C 2 8 Z 0 0 C	SLA D 2 8 Z 0 0 C	SLA E 2 8 Z 0 0 C	SLA H 2 8 Z 0 0 C	SLA L 2 8 Z 0 0 C	SLA [HL] 2 16 Z 0 0 C	SLA A 2 8 Z 0 0 C	SRA B 2 8 Z 0 0 C	SRA C 2 8 Z 0 0 C	SRA D 2 8 Z 0 0 C	SRA E 2 8 Z 0 0 C	SRA H 2 8 Z 0 0 C	SRA L 2 8 Z 0 0 C	SRA [HL] 2 16 Z 0 0 C	SRA A 2 8 Z 0 0 C
3x	SWAP B 2 8 Z 0 0 0	SWAP C 2 8 Z 0 0 0	SWAP D 2 8 Z 0 0 0	SWAP E 2 8 Z 0 0 0	SWAP H 2 8 Z 0 0 0	SWAP L 2 8 Z 0 0 0	SWAP [HL] 2 16 Z 0 0 0	SWAP A 2 8 Z 0 0 0	SRL B 2 8 Z 0 0 C	SRL C 2 8 Z 0 0 C	SRL D 2 8 Z 0 0 C	SRL E 2 8 Z 0 0 C	SRL H 2 8 Z 0 0 C	SRL L 2 8 Z 0 0 C	SRL [HL] 2 16 Z 0 0 C	SRL A 2 8 Z 0 0 C
4x	BIT 0, B 2 8 Z 0 1 -	BIT 0, C 2 8 Z 0 1 -	BIT 0, D 2 8 Z 0 1 -	BIT 0, E 2 8 Z 0 1 -	BIT 0, H 2 8 Z 0 1 -	BIT 0, L 2 8 Z 0 1 -	BIT 0, [HL] 2 12 Z 0 1 -	BIT 0, A 2 8 Z 0 1 -	BIT 1, B 2 8 Z 0 1 -	BIT 1, C 2 8 Z 0 1 -	BIT 1, D 2 8 Z 0 1 -	BIT 1, E 2 8 Z 0 1 -	BIT 1, H 2 8 Z 0 1 -	BIT 1, L 2 8 Z 0 1 -	BIT 1, [HL] 2 12 Z 0 1 -	BIT 1, A 2 8 Z 0 1 -
5x	BIT 2, B 2 8 Z 0 1 -	BIT 2, C 2 8 Z 0 1 -	BIT 2, D 2 8 Z 0 1 -	BIT 2, E 2 8 Z 0 1 -	BIT 2, H 2 8 Z 0 1 -	BIT 2, L 2 8 Z 0 1 -	BIT 2, [HL] 2 12 Z 0 1 -	BIT 2, A 2 8 Z 0 1 -	BIT 3, B 2 8 Z 0 1 -	BIT 3, C 2 8 Z 0 1 -	BIT 3, D 2 8 Z 0 1 -	BIT 3, E 2 8 Z 0 1 -	BIT 3, H 2 8 Z 0 1 -	BIT 3, L 2 8 Z 0 1 -	BIT 3, [HL] 2 12 Z 0 1 -	BIT 3, A 2 8 Z 0 1 -
6x	BIT 4, B 2 8 Z 0 1 -	BIT 4, C 2 8 Z 0 1 -	BIT 4, D 2 8 Z 0 1 -	BIT 4, E 2 8 Z 0 1 -	BIT 4, H 2 8 Z 0 1 -	BIT 4, L 2 8 Z 0 1 -	BIT 4, [HL] 2 12 Z 0 1 -	BIT 4, A 2 8 Z 0 1 -	BIT 5, B 2 8 Z 0 1 -	BIT 5, C 2 8 Z 0 1 -	BIT 5, D 2 8 Z 0 1 -	BIT 5, E 2 8 Z 0 1 -	BIT 5, H 2 8 Z 0 1 -	BIT 5, L 2 8 Z 0 1 -	BIT 5, [HL] 2 12 Z 0 1 -	BIT 5, A 2 8 Z 0 1 -
7x	BIT 6, B 2 8 Z 0 1 -	BIT 6, C 2 8 Z 0 1 -	BIT 6, D 2 8 Z 0 1 -	BIT 6, E 2 8 Z 0 1 -	BIT 6, H 2 8 Z 0 1 -	BIT 6, L 2 8 Z 0 1 -	BIT 6, [HL] 2 12 Z 0 1 -	BIT 6, A 2 8 Z 0 1 -	BIT 7, B 2 8 Z 0 1 -	BIT 7, C 2 8 Z 0 1 -	BIT 7, D 2 8 Z 0 1 -	BIT 7, E 2 8 Z 0 1 -	BIT 7, H 2 8 Z 0 1 -	BIT 7, L 2 8 Z 0 1 -	BIT 7, [HL] 2 12 Z 0 1 -	BIT 7, A 2 8 Z 0 1 -
8x	RES 0, B 2 8 - - - -	RES 0, C 2 8 - - - -	RES 0, D 2 8 - - - -	RES 0, E 2 8 - - - -	RES 0, H 2 8 - - - -	RES 0, L 2 8 - - - -	RES 0, [HL] 2 16 - - - -	RES 0, A 2 8 - - - -	RES 1, B 2 8 - - - -	RES 1, C 2 8 - - - -	RES 1, D 2 8 - - - -	RES 1, E 2 8 - - - -	RES 1, H 2 8 - - - -	RES 1, L 2 8 - - - -	RES 1, [HL] 2 16 - - - -	RES 1, A 2 8 - - - -
9x	RES 2, B 2 8 - - - -	RES 2, C 2 8 - - - -	RES 2, D 2 8 - - - -	RES 2, E 2 8 - - - -	RES 2, H 2 8 - - - -	RES 2, L 2 8 - - - -	RES 2, [HL] 2 16 - - - -	RES 2, A 2 8 - - - -	RES 3, B 2 8 - - - -	RES 3, C 2 8 - - - -	RES 3, D 2 8 - - - -	RES 3, E 2 8 - - - -	RES 3, H 2 8 - - - -	RES 3, L 2 8 - - - -	RES 3, [HL] 2 16 - - - -	RES 3, A 2 8 - - - -
Ax	RES 4, B 2 8 - - - -	RES 4, C 2 8 - - - -	RES 4, D 2 8 - - - -	RES 4, E 2 8 - - - -	RES 4, H 2 8 - - - -	RES 4, L 2 8 - - - -	RES 4, [HL] 2 16 - - - -	RES 4, A 2 8 - - - -	RES 5, B 2 8 - - - -	RES 5, C 2 8 - - - -	RES 5, D 2 8 - - - -	RES 5, E 2 8 - - - -	RES 5, H 2 8 - - - -	RES 5, L 2 8 - - - -	RES 5, [HL] 2 16 - - - -	RES 5, A 2 8 - - - -
Bx	RES 6, B 2 8 - - - -	RES 6, C 2 8 - - - -	RES 6, D 2 8 - - - -	RES 6, E 2 8 - - - -	RES 6, H 2 8 - - - -	RES 6, L 2 8 - - - -	RES 6, [HL] 2 16 - - - -	RES 6, A 2 8 - - - -	RES 7, B 2 8 - - - -	RES 7, C 2 8 - - - -	RES 7, D 2 8 - - - -	RES 7, E 2 8 - - - -	RES 7, H 2 8 - - - -	RES 7, L 2 8 - - - -	RES 7, [HL] 2 16 - - - -	RES 7, A 2 8 - - - -
Cx	SET 0, B 2 8 - - - -	SET 0, C 2 8 - - - -	SET 0, D 2 8 - - - -	SET 0, E 2 8 - - - -	SET 0, H 2 8 - - - -	SET 0, L 2 8 - - - -	SET 0, [HL] 2 16 - - - -	SET 0, A 2 8 - - - -	SET 1, B 2 8 - - - -	SET 1, C 2 8 - - - -	SET 1, D 2 8 - - - -	SET 1, E 2 8 - - - -	SET 1, H 2 8 - - - -	SET 1, L 2 8 - - - -	SET 1, [HL] 2 16 - - - -	SET 1, A 2 8 - - - -
Dx	SET 2, B 2 8 - - - -	SET 2, C 2 8 - - - -	SET 2, D 2 8 - - - -	SET 2, E 2 8 - - - -	SET 2, H 2 8 - - - -	SET 2, L 2 8 - - - -	SET 2, [HL] 2 16 - - - -	SET 2, A 2 8 - - - -	SET 3, B 2 8 - - - -	SET 3, C 2 8 - - - -	SET 3, D 2 8 - - - -	SET 3, E 2 8 - - - -	SET 3, H 2 8 - - - -	SET 3, L 2 8 - - - -	SET 3, [HL] 2 16 - - - -	SET 3, A 2 8 - - - -
Ex	SET 4, B 2 8 - - - -	SET 4, C 2 8 - - - -	SET 4, D 2 8 - - - -	SET 4, E 2 8 - - - -	SET 4, H 2 8 - - - -	SET 4, L 2 8 - - - -	SET 4, [HL] 2 16 - - - -	SET 4, A 2 8 - - - -	SET 5, B 2 8 - - - -	SET 5, C 2 8 - - - -	SET 5, D 2 8 - - - -	SET 5, E 2 8 - - - -	SET 5, H 2 8 - - - -	SET 5, L 2 8 - - - -	SET 5, [HL] 2 16 - - - -	SET 5, A 2 8 - - - -
Fx	SET 6, B 2 8 - - - -	SET 6, C 2 8 - - - -	SET 6, D 2 8 - - - -	SET 6, E 2 8 - - - -	SET 6, H 2 8 - - - -	SET 6, L 2 8 - - - -	SET 6, [HL] 2 16 - - - -	SET 6, A 2 8 - - - -	SET 7, B 2 8 - - - -	SET 7, C 2 8 - - - -	SET 7, D 2 8 - - - -	SET 7, E 2 8 - - - -	SET 7, H 2 8 - - - -	SET 7, L 2 8 - - - -	SET 7, [HL] 2 16 - - - -	SET 7, A 2 8 - - - -

Rysunek 2: Tablica rozmieszczenia instrukcji po opcodzie 0xCB procesora SM83. Źródło: [3]

2.3.4 Instrukcje przesunięć i rotacji

Operacje tej kategorii realizują przesunięcia bitowe oraz rotacje z lub bez udziału flagi przeniesienia:

- **RL, RR** – rotacja w lewo/prawo przez flagę przeniesienia (CY) – bit wypchnięty z rejestru trafia do flagi, a poprzednia wartość flagi wchodzi z drugiej strony,
- **RLC, RRC** – rotacja w lewo/prawo bez udziału flagi przeniesienia – bit wypchnięty wraca na przeciwny koniec rejestru (flaga CY jest jedynie aktualizowana, nie bierze udziału w rotacji),
- **SLA** (Shift Left Arithmetic) – przesuwa bity w lewo. Bit 7 (najstarszy) wypychany jest do flagi CY, a na miejsce bitu 0 wstawiany jest zero. Efekt: mnożenie przez 2.
- **SRA** (Shift Right Arithmetic) – przesuwa bity w prawo. Bit 0 (najmłodszy) wypychany jest do flagi CY, a bit 7 zachowuje swoją poprzednią wartość (zachowany jest bit znaku). Efekt: dzielenie przez 2 z zachowaniem znaku liczby ujemnej.
- **SRL** (Shift Right Logical) – przesuwa bity w prawo. Bit 0 wypychany jest do flagi CY, a na miejsce bitu 7 wstawiany jest zero (bez względu na znak). Efekt: dzielenie przez 2 dla liczb bez znaku.
- **SWAP** – zamiana półbajtów w rejestrze.

Instrukcje te są użyteczne zarówno w obliczeniach arytmetycznych, jak i w przetwarzaniu danych binarnych.

2.3.5 Instrukcje sterowania przepływem programu

Instrukcje sterujące odpowiadają za zmianę kolejności wykonywania programu:

- **JP** – skok bezwarunkowy lub warunkowy, gdzie warunek jest zależny od kodu operacyjnego.
- **JR** – skok względny o ograniczonym zasięgu (Między -128 a +127),
- **CALL / RET** – wywołanie i powrót z podprogramu,
- **RST** – szybkie wywołanie procedury o stałym adresie - często używane w procedurach o dużej uwadze czasowej (obsługa przewań)

Warunkowość operacji zależy od stanu flag procesora, co umożliwia implementację struktur decyzyjnych.

Do tej grupy należą instrukcje wpływające na stan globalny procesora:

- **NOP** – brak operacji,
- **HALT** – zatrzymanie pracy CPU do momentu wystąpienia przerwania,
- **STOP** – tryb niskiego poboru energii,
- **DI / EI** – dezaktywacja i aktywacja systemu przerwań,
- **SCF / CCF** – ustawanie i kasowanie flagi przeniesienia.

Instrukcje te mają kluczowe znaczenie dla zarządzania energią oraz obsługi zdarzeń asynchronicznych.

2.4 Mapowanie pamięci w konsoli Game Boy

Architektura pamięci konsoli Game Boy oparta jest na 16-bitowej przestrzeni adresowej o rozmiarze $2^{16} = 65\,536$ bajtów (64 KB). Procesor systemu, zgodny z modyfikowaną architekturą Sharp LR35902, wykorzystuje jednolitą przestrzeń adresową, w której różne zakresy adresów są przypisane do odmiennych typów pamięci oraz rejestrów sprzętowych. Taki model organizacji pamięci określany jest jako *memory-mapped I/O* i umożliwia bezpośrednią komunikację procesora z układami peryferyjnymi poprzez operacje odczytu i zapisu w odpowiednich obszarach adresowych.

2.4.1 Ogólny układ przestrzeni adresowej

Przestrzeń adresowa Game Boya podzielona jest na segmenty o ustalonej funkcji sprzętowej. Tabela 1 przedstawia standardowy schemat mapowania pamięci.

Zakres adresów	Rozmiar	Przeznaczenie
`0000-3FFF`	16 KB	Stała pamięć ROM banku 0
`4000-7FFF`	16 KB	Przełączalny bank ROM
`8000-9FFF`	8 KB	Pamięć wideo (VRAM)
`A000-BFFF`	8 KB	Zewnętrzna pamięć RAM w kartridżu
`C000-CFFF`	4 KB	Wewnętrzna pamięć robocza (WRAM bank 0)
`D000-DFFF`	4 KB	Wewnętrzna pamięć robocza (bank przełączalny)
`E000-FDFF`	7.5 KB	Obszar echo pamięci WRAM
`FE00-FE9F`	160 B	Pamięć atrybutów obiektów (OAM)
`FEA0-FEFF`	96 B	Obszar niedostępny
`FF00-FF7F`	128 B	Rejestry wejścia/wyjścia
`FF80-FFFE`	127 B	Pamięć szybka (HRAM)
`FFFF`	1 B	Rejestr przerwań (IE)

Tabela 1: Mapa pamięci konsoli Game Boy [9]

2.4.2 Pamięć programu i bankowanie ROM

Dolne 32 KB przestrzeni adresowej przeznaczone jest na pamięć tylko do odczytu (ROM) zawartą w kartridżu. Pierwsze 16 KB (`0000-3FFF`) stanowi bank stały, natomiast zakres `4000-7FFF` obsługuje mechanizm przełączania banków pamięci, realizowany przez kontroler MBC (Memory Bank Controller). Dzięki temu możliwe jest adresowanie programów o rozmiarze znacznie przekraczającym 64 KB.

2.4.3 Pamięć wideo

Obszar `8000-9FFF` odpowiada pamięci VRAM, wykorzystywanej przez układ graficzny do przechowywania danych kafelków graficznych oraz map tła. Dostęp do tej pamięci jest ograniczony w określonych fazach cyklu wyświetlania obrazu, co wynika z synchronizacji procesora z kontrolerem LCD.

2.4.4 Pamięć robocza i obszar echo

Wewnętrzna pamięć robocza WRAM zajmuje zakres `C000-DFFF`. Dodatkowo obszar `E000-FDFF` stanowi tzw. echo WRAM, czyli lustrzane odwzorowanie części pamięci roboczej. Jego obecność wynika z uproszczeń sprzętowych i nie jest zalecana do programowego wykorzystania.

2.4.5 Mapowanie urządzeń wejścia/wyjścia

Zakres `FF00-FF7F` zawiera rejestry sterujące sprzętem, w tym kontrolerem dźwięku, systemem przerwań, timerami oraz interfejsem wejściowym. Operacje zapisu i odczytu w tych adresach odpowiadają bezpośrednim operacjom na urządzeniach peryferyjnych.

2.4.6 Pamięć wysokiej prędkości

Obszar `FF80-FFFE`, określany jako HRAM (High RAM), to niewielki fragment szybkiej pamięci roboczej o krótkim czasie dostępu. Ze względu na swoje właściwości jest on często wykorzystywany do przechowywania zmiennych o krytycznym znaczeniu czasowym.

2.5 System przerwań w konsoli Game Boy

System przerwań w konsoli Game Boy realizuje mechanizm asynchronicznej obsługi zdarzeń sprzętowych, umożliwiając procesorowi reagowanie na zdarzenia wewnętrzne i zewnętrzne bez konieczności ciągłego odpytywania urządzeń peryferyjnych. Architektura ta stanowi kluczowy element synchronizacji pracy jednostki centralnej z układami graficznymi, licznikami czasu oraz interfejsem wejściowym.

2.5.1 Źródła przerwań

Konsola Game Boy obsługuje pięć podstawowych źródeł przerwań sprzętowych. Każdemu z nich przypisany jest odrębny wektor przerwania oraz odpowiedni bit w rejestrach sterujących. Zestaw źródeł przerwań przedstawiono w tabeli 2.

Bit	Nazwa przerwania	Funkcja
0	V-Blank	Zakończenie rysowania ramki obrazu
1	LCD STAT	Zdarzenia kontrolera LCD
2	Timer	Przepełnienie timera systemowego
3	Serial	Zakończenie transmisji szeregowej
4	Joypad	Zmiana stanu kontrolera wejściowego

Tabela 2: Źródła przerwań w konsoli Game Boy[9]

Przerwanie V-Blank pełni szczególną rolę, ponieważ wyznacza bezpieczny moment aktualizacji pamięci wideo. Przerwanie LCD STAT umożliwia reakcję na szczegółowe stany pracy kontrolera wyświetlania, natomiast przerwanie timera wykorzystywane jest do odmierzania czasu oraz synchronizacji logiki programu.

Pozostałe przerwania w systemie Game Boy pochodzą z kontrolera wejścia oraz interfejsu komunikacyjnego. Przerwanie wejścia (Joypad) generowane jest w momencie zmiany stanu przycisków kontrolera, co umożliwia natychmiastową reakcję programu na działania użytkownika. Z kolei przerwanie interfejsu szeregowego (Serial) wyzwalane jest po zakończeniu transmisji danych pomiędzy konsolą a urządzeniem zewnętrznym, na przykład drugą konsolą Game Boy połączoną za pomocą kabla komunikacyjnego.

2.5.2 Rejestry sterujące systemem przerwań

Obsługa przerwań realizowana jest poprzez dwa rejestry mapowane w przestrzeni adresowej:

- `IE` (Interrupt Enable, adres `FFFF`) — określa, które źródła przerwań są aktywne.
- `IF` (Interrupt Flag, adres `FF0F`) — przechowuje informację o zgłoszonych przerwaniach.

Każdy bit w obu rejestrach odpowiada jednemu źródłu przerwania. Zgłoszenie przerwania polega na ustawieniu odpowiedniego bitu w rejestrze `IF`. Jeżeli odpowiadający mu bit w rejestrze `IE` jest ustawiony oraz globalna obsługa przerwań jest włączona, procesor inicjuje procedurę obsługi przerwania.

2.5.3 Mechanizm obsługi przerwania

Po wykryciu aktywnego przerwania procesor wykonuje następującą sekwencję działań:

1. zakończenie bieżącej instrukcji,
2. zapis aktualnej wartości licznika programu na stosie,
3. wyłączenie globalnej obsługi przerwań,
4. skok pod adres wektora przerwania.

Adresy wektorów przerwań są stałe i przypisane do konkretnych źródeł zdarzeń. Po zakończeniu procedury obsługi przerwania wykonywana jest instrukcja powrotu, która przywraca poprzedni stan wykonania programu.

2.5.4 Priorytety przerwań

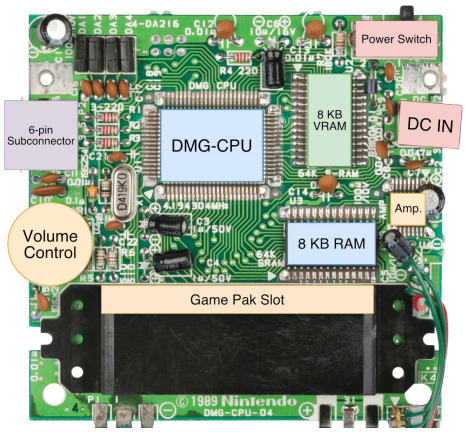
W przypadku jednoczesnego zgłoszenia wielu przerwań stosowany jest ustalony porządek priorytetów. Najwyższy priorytet posiada przerwanie V-Blank, a najniższy przerwanie kontrolera wejściowego. Taki schemat zapewnia deterministyczną obsługę zdarzeń związanych z generowaniem obrazu, które są krytyczne dla poprawnego działania systemu.

2.5.5 Rola systemu przerwań w architekturze konsoli

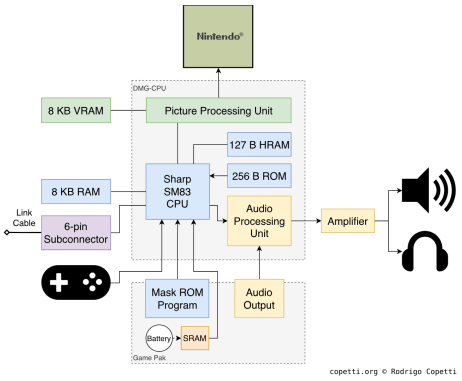
System przerwań stanowi podstawowy mechanizm synchronizacji komponentów sprzętowych konsoli Game Boy. Dzięki niemu możliwe jest efektywne współdziałanie procesora z kontrolerem graficznym, licznikami czasu oraz urządzeniami wejścia/wyjścia przy zachowaniu deterministycznego modelu wykonania programu. Mechanizm ten ma istotne znaczenie zarówno dla projektowania oprogramowania systemowego, jak i dla implementacji emulatorów platformy.

2.6 Płyta główna

Płyta główna konsoli Game Boy stanowi podstawowy element konstrukcyjny systemu, integrujący wszystkie kluczowe układy elektroniczne odpowiedzialne za przetwarzanie danych, generowanie obrazu, obsługę dźwięku oraz zarządzanie energią. Ze względu na przenośny charakter urządzenia, konstrukcja płyty została zaprojektowana z naciskiem na energooszczędność, prostotę architektury oraz wysoką niezawodność.



Rysunek 3: Zdjęcie płyty głównej oryginalnego Game Boy'a z opisami kompomentów[1]



Rysunek 4: Schemat płyty głównej konsoli Game Boy[1]

2.6.1 Jednostka centralna

Centralnym elementem płyty głównej jest układ DMG-CPU, stanowiący zintegrowany system typu System on Chip (SoC). W przeciwieństwie do klasycznych konstrukcji mikrokomputerowych, w których poszczególne funkcje realizowane są przez oddzielne układy scalone, DMG-CPU integruje w jednej strukturze półprzewodnikowej jednostkę obliczeniową, kontroler pamięci, układ graficzny, generator dźwięku oraz logikę sterującą systemem.

Rdzeń obliczeniowy układu bazuje na architekturze 8-bitowej z 16-bitową przestrzenią adresową i realizuje wykonywanie programu oraz zarządzanie przepływem danych pomiędzy komponentami systemu. Integracja funkcji w jednym układzie pozwoliła na ograniczenie liczby połączeń na płycie głównej, zmniejszenie poboru energii oraz zwiększenie niezawodności urządzenia.

DMG-CPU odpowiada również za synchronizację pracy podsystemów, obsługę przerwań sprzętowych, sterowanie dostępem do pamięci operacyjnej i wideo oraz koordynację komunikacji z interfejsem kartridża i urządzeniami wejścia-wyjścia. Zastosowanie architektury SoC stanowiło kluczowy element miniaturyzacji konsoli oraz umożliwiło osiągnięcie wysokiej efektywności energetycznej przy zachowaniu pełnej funkcjonalności systemu.

2.6.2 Pamięć operacyjna i pamięć wideo

Na płycie głównej znajdują się układy pamięci pełniące różne funkcje w architekturze systemu:

- **WRAM (Work RAM)** – pamięć operacyjna wykorzystywana przez procesor do przechowywania danych roboczych oraz stosu.
- **VRAM (Video RAM)** – pamięć przeznaczona do przechowywania danych graficznych, takich jak mapy kafli, wzorce znaków oraz informacje o sprite'ach.

Pamięci te współpracują bezpośrednio z procesorem oraz układem generowania obrazu, zapewniając sprawną wymianę danych niezbędnych do działania systemu.

2.6.3 Układ graficzny

Funkcje generowania obrazu realizowane są przez zintegrowany kontroler LCD, który odpowiada za przetwarzanie danych graficznych i sterowanie wyświetlaczem ciekłokrystalicznym. Układ ten obsługuje system kafli (tile-based graphics), renderowanie sprite'ów oraz synchronizację obrazu z

odświeżaniem ekranu. Kontroler LCD współpracuje bezpośrednio z pamięcią wideo oraz procesorem, tworząc kompletny tor przetwarzania grafiki.

2.6.4 Układ dźwiękowy

System dźwiękowy konsoli jest zintegrowany z architekturą płyty głównej i składa się z generatorów fal dźwiękowych sterowanych programowo. Układ ten umożliwia generowanie kilku kanałów audio, w tym fal prostokątnych, szumu oraz próbek o ograniczonej rozdzielczości. Sygnał audio kierowany jest następnie do wzmacniacza i głośnika lub wyjścia słuchawkowego. Ze względu skomplikowania, kompoment układu dźwiękowego nie będzie uwzględniany w implementacji.

2.6.5 Interfejs kartridża

Na płycie głównej znajduje się złącze kartridża umożliwiające komunikację z zewnętrznym nośnikiem oprogramowania. Interfejs ten zapewnia:

- dostęp do pamięci programu,
- komunikację z dodatkowymi układami w kartridżu,
- możliwość rozszerzenia funkcjonalności systemu.

Zastosowanie wymiennych kartridży umożliwiło modularną architekturę systemu oraz łatwą dystrybucję oprogramowania.

2.6.6 Układy wejścia i wyjścia

Płyta główna integruje również komponenty odpowiedzialne za komunikację z użytkownikiem oraz otoczeniem:

- kontroler przycisków sterujących,
- interfejs komunikacyjny (port szeregowy),
- sterowanie wyświetlaczem LCD,
- wzmacniacz audio.

Układy te zapewniają interakcję użytkownika z systemem oraz obsługę sygnałów wejściowych i wyjściowych.

Płyta główna konsoli Game Boy stanowi przykład zwartej i funkcjonalnej integracji elementów elektronicznych w systemie o ograniczonych zasobach sprzętowych. Zastosowanie zintegrowanych układów scalonych, zoptymalizowane zarządzanie energią oraz modułowa struktura pamięci i interfejsów umożliwiły stworzenie wydajnego, przenośnego systemu rozrywkowego. Analiza budowy płyty głównej pozwala dostrzec kompromis pomiędzy funkcjonalnością, energooszczędnością a prostotą konstrukcji, charakterystyczny dla projektowania urządzeń przenośnych końca XX wieku.

2.7 Kartridże systemu Game Boy

Kartridże stanowią podstawowy nośnik oprogramowania dla konsoli Game Boy oraz element rozszerzający funkcjonalność systemu. Oprócz pamięci programu zawierają one układy logiczne odpowiedzialne za zarządzanie pamięcią, zapisywanie danych użytkownika oraz, w niektórych przypadkach, dodatkowe funkcje sprzętowe. Różnorodność konstrukcji kartridżów wynikała z konieczności obsługi gier o rosnącej złożoności przy zachowaniu ograniczeń architektury systemu.

2.7.1 Kartridże bez kontrolera banków pamięci (ROM Only)

Najprostszy typ kartridża zawiera wyłącznie pamięć ROM z programem gry. Konstrukcja ta nie umożliwia przełączania banków pamięci ani zapisu danych użytkownika. Ze względu na ograniczoną pojemność stosowana była głównie w prostszych produkcjach we wczesnym okresie rozwoju systemu.

2.7.2 Kartridże z kontrolerem MBC1

Kontroler Memory Bank Controller 1 (MBC1) umożliwia przełączanie banków pamięci ROM oraz, opcjonalnie, banków pamięci RAM. Układ ten pozwalał na znaczące zwiększenie maksymalnego rozmiaru programu. Kartridże tego typu występowały w kilku wariantach:

- MBC1 + RAM,
- MBC1 + RAM + bateria podtrzymująca zapis danych.

2.7.3 Kartridże z kontrolerem MBC2

MBC2 integruje funkcję przełączania banków ROM z niewielką, wbudowaną pamięcią RAM przeznaczoną do zapisu danych gry. W odróżnieniu od innych kontrolerów, pamięć RAM jest integralną częścią układu i nie występuje jako oddzielny komponent. Kartridże te standardowo wyposażone były w baterię podtrzymującą zapis.

2.7.4 Kartridże z kontrolerem MBC3

MBC3 rozszerza funkcjonalność poprzednich kontrolerów o zegar czasu rzeczywistego (RTC). Rozwiązanie to umożliwiało implementację mechanik zależnych od upływu czasu rzeczywistego. Warianty konstrukcyjne obejmowały:

- MBC3 + RAM,
- MBC3 + RAM + bateria,
- MBC3 + RAM + bateria + RTC.

2.7.5 Kartriże z kontrolerem MBC4

W dokumentacji platformy spotykana jest również wzmianka o kontrolerze MBC4. Jednak dostępne źródła nie dostarczają jednoznacznych dowodów na jego wykorzystanie w komercyjnych kartridżach Game Boy. Z tego względu w wielu opracowaniach przyjmuje się, że układ ten nie został wykorzystany w praktyce lub jego zastosowanie było bardzo ograniczone.

2.7.6 Kartridże z kontrolerem MBC5

MBC5 wprowadza rozszerzony system adresowania pamięci ROM oraz obsługę większych pojemności pamięci RAM. Układ ten był stosowany w późniejszych produkcjach o dużej objętości danych. Warianty obejmowały:

- MBC5 + RAM,
- MBC5 + RAM + bateria,
- MBC5 + RAM + bateria + silnik wibracyjny (Rumble).

2.7.7 Kartridże specjalizowane

Oprócz standardowych kontrolerów banków pamięci produkowano kartridże wyposażone w dodatkowe układy funkcjonalne rozszerzające możliwości systemu. Do najważniejszych należą:

- kartridże z czujnikiem ruchu,
- kartridże z pamięcią EEPROM,
- kartridże z dodatkowymi układami logicznymi sterującymi akcesoriami,
- kartridże diagnostyczne i testowe wykorzystywane w procesie produkcyjnym.

2.7.8 Struktura fizyczna kartridża

Typowy kartridż składa się z płytki drukowanej zawierającej układ ROM, opcjonalną pamięć RAM, kontroler banków pamięci oraz element podtrzymania zasilania pamięci zapisu. Komunikacja z konsolą odbywa się za pośrednictwem złącza krawędziowego, które zapewnia dostęp do magistrali adresowej, magistrali danych oraz linii sterujących.

3 Model działania systemu Game Boy

3.1 Inicjalizacja systemu

Proces inicjalizacji systemu rozpoczyna się bezpośrednio po podaniu zasilania i stanowi sekwencję operacji przygotowujących komponenty sprzętowe do poprawnego wykonywania programu użytkownika. Etap ten obejmuje ustawienie stanu początkowego procesora, konfigurację rejestrów sterujących, inicjalizację pamięci oraz przygotowanie układu graficznego do generowania obrazu.

3.1.1 Stan początkowy procesora

Po uruchomieniu systemu procesor rozpoczyna wykonywanie kodu spod zdefiniowanego adresu startowego. Rejestry procesora oraz wskaźniki systemowe przyjmują wartości początkowe określone przez logikę sprzętową. W tym etapie przygotowywany jest stos, a licznik programu kierowany jest do procedury inicjalizacyjnej.

W początkowej fazie globalny mechanizm obsługi przerwań pozostaje wyłączony, co zapobiega zakłóceniu procesu konfiguracji systemu przez zdarzenia asynchroniczne.

3.1.2 Konfiguracja pamięci i przestrzeni adresowej

Procedura inicjalizacji obejmuje przygotowanie obszarów pamięci wykorzystywanych przez system. Dane w pamięci operacyjnej mogą zostać wyzerowane lub ustawione na wartości domyślne, a rejestry sterujące urządzeń peryferyjnych otrzymują konfigurację początkową.

Istotnym elementem jest także przygotowanie struktur danych wykorzystywanych przez układ graficzny, w szczególności map tła oraz danych obiektów. W tym etapie możliwe jest również przygotowanie obszarów buforowych przeznaczonych do późniejszego wykorzystania przez program użytkownika.

3.1.3 Inicjalizacja układu graficznego

Układ graficzny zostaje skonfigurowany poprzez zapis odpowiednich wartości do rejestrów sterujących. Konfiguracja obejmuje włączenie wyświetlania obrazu, ustawienie parametrów renderowania tła oraz przygotowanie pamięci przechowującej dane graficzne.

W początkowej fazie generowanie obrazu może być czasowo wstrzymane, co umożliwia bezpieczne przygotowanie danych w pamięci wideo oraz pamięci atrybutów obiektów. Po zakończeniu konfiguracji układ graficzny przechodzi do trybu normalnej pracy i rozpoczyna cykliczne generowanie klatek obrazu.

3.1.4 Przygotowanie systemu przerwań

Po zakończeniu konfiguracji podstawowych komponentów system aktywuje wybrane źródła przerwań. Rejestr maskujący przerwania zostaje ustawiony zgodnie z wymaganiami programu, a następnie globalna obsługa przerwań może zostać włączona.

Aktywacja mechanizmu przerwań umożliwia synchronizację działania programu z układami peryferyjnymi oraz reagowanie na zdarzenia zachodzące w systemie w czasie rzeczywistym.

3.1.5 Przekazanie sterowania programowi użytkownika

Końcowym etapem inicjalizacji jest przekazanie sterowania właściwemu programowi wykonywalnemu. Od tego momentu system pracuje w trybie operacyjnym, a dalsze działanie konsoli jest determinowane przez kod programu oraz zdarzenia sprzętowe.

3.1.6 Znaczenie procesu inicjalizacji

Poprawna inicjalizacja systemu stanowi warunek konieczny stabilnego działania wszystkich komponentów sprzętowych. Sekwencja operacji wykonywanych po uruchomieniu zapewnia spójność konfiguracji urządzeń, deterministyczny stan początkowy systemu oraz przygotowanie środowiska wykonawczego dla programu użytkownika.

3.2 Cykl pracy procesora

Procesor zastosowany w konsoli Game Boy jest 8-bitową jednostką opartą na zmodyfikowanej architekturze zbliżonej do Z80. Jego działanie jest determinowane przez cykl zegara systemowego oraz mechanizm wykonywania instrukcji w modelu sekwencyjnym. Zrozumienie cyklu pracy procesora jest kluczowe dla analizy działania całego systemu, ponieważ CPU koordynuje komunikację pomiędzy pamięcią, układem graficznym oraz urządzeniami wejścia/wyjścia.

3.2.1 Model cyklu rozkazowego

Podstawowy cykl pracy procesora można opisać jako powtarzającą się sekwencję trzech etapów:

1. pobranie instrukcji z pamięci,
2. dekodowanie instrukcji,
3. wykonanie instrukcji.

W pierwszym etapie procesor odczytuje kod operacji spod adresu wskazywanego przez licznik programu (PC). Następnie licznik programu zostaje inkrementowany, a pobrana wartość trafia do wewnętrznego rejestru instrukcji.

W drugim etapie następuje interpretacja kodu operacji oraz ustalenie wymaganych operandów. Proces dekodowania determinuje, czy instrukcja wymaga dodatkowych danych z pamięci oraz jakie jednostki funkcjonalne procesora zostaną zaangażowane w jej wykonanie.

Trzeci etap obejmuje właściwe wykonanie operacji arytmetycznej, logicznej lub sterującej. W zależności od typu instrukcji może to obejmować operacje na rejestrach, dostęp do pamięci lub zmianę przepływu sterowania programu.

3.2.2 Synchronizacja z zegarem systemowym

Procesor pracuje synchronicznie z zegarem systemowym, którego częstotliwość determinuje tempo wykonywania operacji. Każda instrukcja wymaga określonej liczby cykli zegara do wykonania. Czas realizacji instrukcji nie jest stały i zależy od jej złożoności oraz liczby operacji pamięciowych.

Istotnym elementem synchronizacji jest współdzielenie magistrali danych z innymi komponentami systemu. Dostęp do pamięci jest realizowany w ściśle określonych momentach cyklu zegara, co zapewnia spójność transmisji danych pomiędzy CPU a pozostałymi układami.

3.2.3 Dostęp do pamięci i operacje magistrali

Procesor komunikuje się z pamięcią poprzez magistralę adresową i magistralę danych. W trakcie cyklu rozkazowego mogą wystąpić operacje odczytu lub zapisu pamięci. Operacje te są wykonywane sekwencyjnie i blokują dostęp do magistrali dla innych komponentów na czas trwania transferu.

Mapowanie pamięci odgrywa istotną rolę w cyklu pracy procesora, ponieważ różne zakresy adresowe odpowiadają różnym typom zasobów systemowych, takim jak pamięć operacyjna, pamięć wideo oraz rejestry sterujące urządzeń peryferyjnych.

3.2.4 Obsługa przerwań

Cykl pracy procesora może zostać przerwany przez mechanizm przerwań sprzętowych. Wystąpienie przerwania powoduje czasowe wstrzymanie wykonywania bieżącego programu oraz zapis aktualnego stanu procesora na stosie.

Po zaakceptowaniu przerwania procesor przechodzi do procedury obsługi przerwania wskazanej w tablicy wektorów przerwań. Po zakończeniu obsługi następuje przywrócenie poprzedniego stanu procesora i kontynuacja wykonywania programu od miejsca przerwania.

3.2.5 Znaczenie cyklu pracy dla działania systemu

Deterministyczny charakter cyklu pracy procesora umożliwia precyzyjną synchronizację z układem graficznym oraz pozostałymi komponentami systemu. W szczególności liczba cykli zegara zużywana przez instrukcje wpływa bezpośrednio na generowanie obrazu oraz obsługę zdarzeń wejścia/wyjścia.

Analiza cyklu pracy procesora stanowi podstawę do modelowania działania systemu oraz jest niezbędna przy implementacji oprogramowania emulującego działanie konsoli.

3.3 Magistrala pamięciowa i charakterystyka działania pamięci

System pamięci konsoli Game Boy opiera się na wspólnej przestrzeni adresowej obsługiwanej przez procesor oraz na zestawie wyspecjalizowanych obszarów pamięci o odmiennych funkcjach. Komunikacja pomiędzy jednostką centralną a zasobami pamięciowymi realizowana jest za pomocą magistrali adresowej, magistrali danych oraz sygnałów sterujących, które determinują kierunek i tryb transferu informacji.

3.3.1 Organizacja magistrali pamięciowej

Magistrala adresowa umożliwia wskazanie konkretnej komórki pamięci w przestrzeni adresowej systemu. Procesor generuje adres, który identyfikuje zasób, z którego ma nastąpić odczyt danych lub do którego ma zostać wykonany zapis. Magistrala danych służy do przesyłania właściwych informacji pomiędzy procesorem a wskazanym komponentem systemu.

Operacje pamięciowe są synchronizowane z zegarem systemowym. W każdym cyklu dostępu do pamięci następuje ustalenie adresu, aktywacja sygnału sterującego oraz transfer danych. Ze względu na współdzielenie magistrali przez różne komponenty systemu dostęp do niej jest ściśle kontrolowany, co zapobiega konfliktom transmisji.

3.3.2 Mapowanie przestrzeni adresowej

Przestrzeń adresowa systemu jest podzielona na logiczne segmenty odpowiadające różnym typom zasobów sprzętowych. Poszczególne zakresy adresów są przypisane do pamięci programu, pamięci operacyjnej, pamięci wideo oraz rejestrów sterujących urządzeń peryferyjnych.

Mapowanie pamięci pełni funkcję mechanizmu integrującego różnorodne komponenty sprzętowe w jednolitą przestrzeń adresową. Dzięki temu procesor może komunikować się z układami systemowymi za pomocą standardowych operacji odczytu i zapisu, bez konieczności stosowania odrębnych instrukcji wejścia/wyjścia.

3.3.3 Rodzaje pamięci w systemie

W architekturze systemu można wyróżnić kilka podstawowych typów pamięci:

- pamięć programu (ROM), zawierająca kod wykonywalny oraz dane statyczne,
- pamięć operacyjna (RAM), wykorzystywana do przechowywania danych tymczasowych,
- pamięć wideo (VRAM), przeznaczona do przechowywania danych graficznych,
- rejestry urządzeń peryferyjnych odwzorowane w przestrzeni adresowej.

Każdy z wymienionych typów pamięci charakteryzuje się odmiennym przeznaczeniem funkcjonalnym oraz sposobem dostępu. W szczególności pamięć wideo jest współdzielona z układem graficznym, co wprowadza ograniczenia czasowe w dostępie do niej przez procesor.

3.3.4 Charakterystyka operacji pamięciowych

Operacje pamięciowe wykonywane przez procesor obejmują odczyt danych, zapis danych oraz transfery blokowe realizowane sekwencyjnie. Czas trwania operacji zależy od rodzaju pamięci oraz liczby cykli zegara wymaganych do realizacji transferu.

Dostęp do pamięci może być ograniczony przez stan systemu, w szczególności przez aktywność układów współdzielących magistralę. W określonych fazach pracy systemu wybrane obszary pamięci mogą być czasowo niedostępne dla procesora lub dostępne jedynie w określonych warunkach synchronizacji.

3.3.5 Rola systemu pamięci w działaniu konsoli

Architektura pamięci stanowi podstawę wymiany informacji pomiędzy wszystkimi komponentami systemu. Determinuje ona sposób wykonywania programu, obsługi grafiki oraz komunikacji z urządzeniami wejścia/wyjścia.

Jednolita przestrzeń adresowa oraz deterministyczny model dostępu do pamięci umożliwiają przewidywalne działanie systemu w czasie rzeczywistym. Właściwości te mają istotne znaczenie zarówno

dla analizy architektury sprzętowej, jak i dla implementacji oprogramowania odwzorowującego działanie systemu.

3.4 Generowanie klatki obrazu

Generowanie obrazu w konsoli Game Boy jest realizowane przez układ graficzny (PPU), który współpracuje z pamięcią wideo oraz pamięcią atrybutów obiektów. Proces ten przebiega cyklicznie i jest ściśle zsynchronizowany z zegarem systemowym oraz działaniem procesora. Obraz powstaje poprzez sekwencyjne renderowanie tła, a następnie nakładanie obiektów ruchomych zapisanych w pamięci OAM (Rozdział 3.4.3).

3.4.1 Format zapisu grafiki w pamięci VRAM

Podstawową jednostką reprezentacji obrazu w systemie jest tzw. kafel (ang. tile). Każdy kafel reprezentuje fragment obrazu o wymiarach 8×8 pikseli. Dane kafli przechowywane są w pamięci wideo (VRAM) w postaci uporządkowanej tablicy bajtów.

Każdy wiersz kafla kodowany jest przy użyciu dwóch bajtów, które reprezentują dwubitową informację o kolorze każdego piksela w wierszu. Pierwszy bajt zawiera najmniej znaczące bity wartości koloru, natomiast drugi bajt zawiera bity najbardziej znaczące. Odczyt odpowiadających sobie bitów z obu bajtów pozwala określić indeks koloru dla pojedynczego piksela.

Układ graficzny interpretuje dane kafli na podstawie mapy tła, która przechowuje indeksy kafli oraz ich rozmieszczenie na ekranie. Dzięki temu możliwe jest wielokrotne wykorzystanie tych samych danych graficznych w różnych miejscach obrazu.

3.4.2 Renderowanie tła

Proces generowania klatki rozpoczyna się od renderowania tła. Układ graficzny przetwarza mapę tła linia po linii, odczytując odpowiednie indeksy kafli oraz ich dane z pamięci VRAM. Dla każdej linii obrazu wyznaczany jest odpowiedni fragment kafla, który następnie przekształcany jest w wartości kolorów pikseli.

Renderowanie odbywa się w sposób deterministyczny i jest zsynchronizowane z aktualną pozycją skanowania ekranu. W wyniku tego procesu powstaje podstawowa warstwa obrazu, która stanowi bazę dla dalszego etapu renderowania.

3.4.3 Pamięć OAM i reprezentacja obiektów

OAM (Object Attribute Memory) jest wyspecjalizowanym obszarem pamięci przeznaczonym do przechowywania informacji o obiektach ruchomych, określanych jako sprite'y. Pamięć ta zawiera zestaw rekordów opisujących właściwości graficzne oraz położenie obiektów wyświetlanych na ekranie.

Każdy wpis w pamięci OAM zawiera:

- współrzędną pionową obiektu na ekranie,
- współrzędną poziomą obiektu,
- indeks kafla opisującego grafikę obiektu,
- zestaw atrybutów określających sposób renderowania.

Atrybuty obiektu mogą obejmować informacje o priorytecie wyświetlania, odbiciu lustrzanym oraz wyborze palety kolorów. Układ graficzny odczytuje zawartość pamięci OAM podczas procesu generowania obrazu i wykorzystuje ją do nałożenia obiektów na wcześniej wyrenderowane tło.

3.4.4 Renderowanie obiektów

Po wygenerowaniu warstwy tła układ graficzny przystępuje do renderowania obiektów zapisanych w pamięci OAM. Dla każdej linii obrazu wybierane są obiekty znajdujące się w jej obszarze, a następnie ich dane graficzne są pobierane z pamięci VRAM.

Renderowanie obiektów polega na nałożeniu ich pikseli na istniejący obraz tła zgodnie z regułami priorytetu. W przypadku kolizji pikseli o różnych źródłach decyzja o widoczności zależy od atrybutów obiektu oraz wartości koloru tła.

3.4.5 DMA i bezpośredni transfer danych do OAM

W celu usprawnienia transferu danych do pamięci OAM system wykorzystuje mechanizm DMA (Direct Memory Access). Mechanizm ten umożliwia bezpośrednie kopiowanie bloków danych z określonego obszaru pamięci do pamięci OAM bez konieczności wykonywania instrukcji kopiowania przez procesor.

Transfer DMA polega na automatycznym odczycie kolejnych bajtów ze źródłowego obszaru pamięci oraz ich zapisie do kolejnych komórek pamięci OAM. W trakcie operacji DMA procesor ma ograniczony dostęp do magistrali pamięci, co zapewnia spójność przesyłanych danych.

Zastosowanie mechanizmu bezpośredniego transferu bajtów umożliwia szybkie aktualizowanie danych obiektów, co jest szczególnie istotne w systemach czasu rzeczywistego, gdzie konieczna jest synchronizacja zmian graficznych z generowaniem kolejnych klatek obrazu.

3.5 System przerwań

Mechanizm przerwań w konsoli Game Boy umożliwia asynchroniczną reakcję procesora na zdarzenia sprzętowe zachodzące w systemie. Zamiast ciągłego odpytywania stanu urządzeń peryferyjnych, procesor może wykonywać program główny, a w momencie wystąpienia określonego zdarzenia przejść do dedykowanej procedury obsługi przerwania. Rozwiązanie to zwiększa efektywność wykorzystania czasu procesora oraz zapewnia deterministyczną obsługę zdarzeń czasowych.

3.5.1 Warunki występowania przerwań

Przerwanie może zostać wygenerowane przez układ sprzętowy po spełnieniu określonego warunku logicznego, takiego jak zakończenie operacji wejścia/wyjścia, osiągnięcie zadanego stanu licznika czasowego lub zmiana stanu kontrolera wejściowego. Aby przerwanie zostało obsłużone, muszą zostać spełnione trzy warunki:

1. źródło przerwania zgłosi żądanie,
2. odpowiedni typ przerwania jest odblokowany w rejestrze maskującym,
3. globalny mechanizm obsługi przerwań w procesorze jest aktywny.

Jeżeli wszystkie warunki są spełnione, procesor kończy wykonywanie bieżącej instrukcji, zapisuje aktualny stan wykonania programu oraz przechodzi do procedury obsługi przerwania.

3.5.2 Rodzaje przerwań

Każdy typ przerwania (tabela 2) posiada przypisany wektor, czyli stały adres w pamięci programu, pod którym rozpoczyna się procedura jego obsługi. Priorytet przerwań jest ustalony sprzętowo i determinuje kolejność ich obsługi w przypadku jednoczesnego wystąpienia kilku zdarzeń.

3.5.3 Przebieg obsługi przerwania

Obsługa przerwania przebiega według ściśle określonej sekwencji działań wykonywanych przez procesor:

1. zakończenie wykonywania bieżącej instrukcji,
2. zapis aktualnej wartości licznika programu na stosie,
3. czasowe zablokowanie dalszych przerwań,
4. przejście do adresu procedury obsługi przerwania,
5. wykonanie kodu obsługi zdarzenia,
6. przywrócenie poprzedniego stanu programu i kontynuacja wykonania.

Zapis stanu programu na stosie umożliwia powrót do przerwanego miejsca po zakończeniu obsługi. Instrukcja powrotu z przerwania przywraca licznik programu oraz ponownie aktywuje globalny mechanizm przerwań.

3.5.4 Funkcje realizowane przez przerwania

Przerwania pełnią w systemie rolę mechanizmu synchronizującego działanie procesora z procesami zachodzącymi w układach peryferyjnych. Do najważniejszych funkcji realizowanych za ich pomocą należą:

- synchronizacja zmian graficznych z cyklem generowania obrazu,
- odmierzanie czasu oraz generowanie zdarzeń okresowych,
- obsługa transmisji danych pomiędzy komponentami systemu,
- reagowanie na działania użytkownika.

W szczególności przerwanie synchronizacji pionowej umożliwia bezpieczną aktualizację danych graficznych w pamięci wideo w momentach, gdy układ graficzny nie odczytuje danych do generowania obrazu. Mechanizm przerwań zapewnia tym samym spójność danych oraz stabilność działania systemu w czasie rzeczywistym.

3.5.5 Znaczenie mechanizmu przerwań

Zastosowanie przerwań pozwala na efektywne zarządzanie współbieżnymi zdarzeniami w systemie o ograniczonych zasobach sprzętowych. Deterministyczna obsługa zdarzeń oraz priorytetyzacja źródeł przerwań stanowią podstawę prawidłowego działania konsoli oraz umożliwiają implementację oprogramowania wymagającego precyzyjnej synchronizacji czasowej.

4 Implementacja

4.1 Wykorzystane narzędzia i zasoby

Do implementacji emulatora zastosowano język C++[14] ze względu na jego niskopoziomową naturę, wraz z bibliotekami Raylib w wersji 5.0 [13] dla implementancji graficznej i sterowania oraz fmt w wersji 10.2.1 [16] dla łatwiejszego formatowania tekstu.

4.1.1 Funkcje biblioteki Raylib

Funkcja `InitWindow()` inicjalizuje okno aplikacji oraz kontekst renderowania, umożliwiając rozpoczęcie operacji graficznych. Cykl renderowania pojedynczej klatki realizowany jest przy użyciu funkcji `BeginDrawing()` oraz `EndDrawing()`, pomiędzy którymi wykonywane są wszystkie operacje rysowania.

Do rysowania elementów graficznych wykorzystywane są następujące funkcje:

- `ClearBackground()` – czyści bufor renderowania i wypełnia go zadanym kolorem,
- `DrawText()` – umożliwia prezentację tekstu,
- `DrawRectangle()` – rysuje wypełniony prostokąt,
- `DrawLine()` – rysuje linię,
- `DrawRectangleRec()` – rysuje prostokąt na podstawie struktury `Rectangle`,
- `DrawRectangleLines()` – rysuje obrys prostokąta,
- `DrawRectangleLinesEx()` – rysuje obrys prostokąta z kontrolą grubości linii.

Renderowanie tekstur realizowane jest przy użyciu:

- `DrawTexture()` – rysuje teksturę w zadanej pozycji,
- `DrawTextureEx()` – rysuje teksturę z dodatkowymi transformacjami geometrycznymi, takimi jak skalowanie i obrót,
- `UpdateTexture()` – aktualizuje zawartość tekstury w pamięci karty graficznej.

Operacje na danych obrazu w pamięci operacyjnej realizowane są przez:

- `ImageClearBackground()` – wypełnia obraz zadanym kolorem,
- `ImageDrawPixel()` – umożliwia bezpośrednią modyfikację pojedynczych pikseli przed przesłaniem ich do tekstury.

Obsługa wejścia użytkownika realizowana jest przez:

- `IsMouseButtonPressed()` – wykrywa naciśnięcie przycisku myszy w bieżącej klatce,
- `IsKeyPressed()` – wykrywa naciśnięcie klawisza klawiatury w bieżącej klatce.

Konfigurację działania biblioteki umożliwiają:

- `SetTargetFPS()` – ustala docelową częstotliwość generowania klatek,
- `SetTraceLogLevel()` – konfiguruje poziom szczegółowości komunikatów diagnostycznych.

4.1.2 Struktury biblioteki Raylib

Biblioteka Raylib definiuje następujące struktury danych:

- `Color` – reprezentuje kolor w przestrzeni RGBA, zawiera składowe intensywności poszczególnych barw oraz przezroczystości,
- `Image` – przechowuje dane obrazu w pamięci operacyjnej, w tym rozdzielczość, format pikseli oraz bufor danych graficznych,
- `Texture` – reprezentuje teksturę przechowywaną w pamięci karty graficznej, wykorzystywaną w procesie renderowania.

4.1.3 Funkcje biblioteki fmt

Biblioteka `fmt` dostarcza następujące funkcje formatowania tekstu:

- `fmt::print()` – wyświetla sformatowany tekst na standardowym wyjściu,
- `fmt::println()` – wyświetla sformatowany tekst zakończony znakiem nowej linii,
- `fmt::format()` – zwraca sformatowany ciąg znaków bez jego wyświetlania, umożliwiając dalsze przetwarzanie tekstu w programie.

4.2 Implementacja modułu obsługi kartridża

Moduł kartridża odpowiada za wczytanie obrazu ROM z pliku binarnego, interpretację jego nagłówka oraz udostępnienie danych pamięci programu pozostałym komponentom emulatora. Implementacja została wykonana w języku C++ z wykorzystaniem standardowej biblioteki wejścia–wyjścia.

Stan kartridża przechowywany jest w strukturze `cart_context`, która zawiera rozmiar pamięci ROM, wskaźnik na dane ROM, wskaźnik na strukturę nagłówka oraz nazwę pliku źródłowego. Dane ROM są ładowane do dynamicznie alokowanego bufora pamięci, natomiast nagłówek interpretowany jest poprzez rzutowanie fragmentu bufora na strukturę `rom_header`. Zgodnie ze specyfikacją formatu ROM, nagłówek znajduje się pod adresem `0x100`.

4.2.1 Struktura nagłówka ROM

Informacje identyfikacyjne oraz parametry konfiguracyjne kartridża przechowywane są w nagłówku ROM. W implementacji nagłówek odwzorowano bezpośrednio jako strukturę języka C++, której układ odpowiada rozmieszczeniu danych w obrazie pamięci kartridża.

Struktura `rom_header` zawiera następujące pola:

```
struct rom_header {
    uint8_t  entry[4];            // kod startowy
    uint8_t  nintendo_logo[0x30]; // dane graficzne logo producenta
    char     title[16];           // nazwa programu
    uint8_t  license_code;        // kod producenta
    uint8_t  new_license_code;    // rozszerzony kod producenta
    uint8_t  sgb_flag;            // flaga zgodnosci ze sprzetem
    uint8_t  type;                // typ kartridza i kontroler pamieci
    uint8_t  rom_size;            // rozmiar pamieci ROM
    uint8_t  ram_size;            // rozmiar pamieci RAM kartridza
    uint8_t  dest_code;           // kod regionu dystrybucji
    uint8_t  version;             // wersja programu
    uint8_t  checksum;            // suma kontrolna naglowka
    uint16_t global_checksum;     // suma kontrolna calego obrazu ROM
};
```

Listing 1: Struktura `rom_header`

Nagłówek znajduje się w pamięci ROM pod stałym adresem `0x100`. W implementacji dostęp do niego realizowany jest poprzez rzutowanie odpowiedniego fragmentu bufora pamięci na strukturę `rom_header`. Takie podejście umożliwia bezpośredni odczyt pól nagłówka bez konieczności ręcznej interpretacji poszczególnych bajtów.

Interfejs dostępu Moduł kartridża udostępnia funkcję `get_rom_header()`, która zwraca wskaźnik na strukturę nagłówka. Pozwala to pozostałym komponentom emulatora na odczyt parametrów kartridża bez konieczności bezpośredniego dostępu do bufora pamięci ROM.

4.2.2 Proces ładowania obrazu ROM

Funkcja `cart_load()` realizuje proces inicjalizacji kartridża. W pierwszym etapie plik otwierany jest w trybie binarnym, a następnie jego zawartość wczytywana jest w całości do pamięci operacyjnej. Po załadowaniu danych wyznaczany jest wskaźnik na strukturę nagłówka, z której odczytywane są podstawowe informacje identyfikacyjne.

W trakcie inicjalizacji prezentowane są następujące parametry kartridża:

- tytuł programu,
- typ kartridża,
- rozmiar pamięci ROM,
- rozmiar pamięci RAM,
- kod licencyjny producenta,
- wersja ROM.

Interpretacja typu kartridża oraz kodu licencyjnego realizowana jest przy użyciu statycznych map odwzorowujących identyfikatory numeryczne na ich reprezentacje tekstowe.

4.2.3 Dostęp do danych kartridża

Moduł udostępnia interfejs umożliwiający dostęp do danych pamięci ROM przez pozostałe komponenty emulatora. Funkcja `cart_read(uint16_t address)` zwraca wartość bajtu znajdującego się pod wskazanym adresem pamięci kartridża. Dodatkowo moduł udostępnia funkcję `get_rom_header()`, umożliwiającą odczyt informacji nagłówkowych przez inne podsystemy emulatora.

Aktualna implementacja obsługuje wyłącznie kartridże typu `ROM ONLY`, bez mechanizmu bankowania pamięci.

4.2.4 Weryfikacja poprawności nagłówka

Poprawność nagłówka weryfikowana jest poprzez obliczenie sumy kontrolnej na podstawie danych znajdujących się w zakresie adresów `0x0134-0x014C`. Obliczona wartość porównywana jest z sumą kontrolną zapisaną w nagłówku ROM. Wynik weryfikacji prezentowany jest w postaci komunikatu diagnostycznego podczas inicjalizacji kartridża.

4.2.5 Ograniczenia implementacji

Obecna wersja modułu stanowi uproszczoną implementację mechanizmu obsługi kartridża. Nie zaimplementowano mechanizmów bankowania pamięci ani obsługi dodatkowych kontrolerów pamięci (MBC). Przyjęte podejście umożliwia jednak poprawne działanie programów wykorzystujących podstawowy model pamięci ROM.

4.3 Implementacja magistrali systemowej

Magistrala systemowa stanowi warstwę pośrednią pomiędzy procesorem a komponentami sprzętowymi emulatora. Jej zadaniem jest odwzorowanie przestrzeni adresowej systemu oraz przekierowanie operacji odczytu i zapisu do właściwych modułów funkcjonalnych. Implementacja magistrali realizuje zatem logiczne mapowanie adresów pamięci na odpowiadające im urządzenia peryferyjne lub obszary pamięci.

4.3.1 Operacje odczytu z magistrali

Podstawową funkcją magistrali jest procedura `bus_read(uint16_t address)`, która realizuje odczyt pojedynczego bajtu z przestrzeni adresowej. Funkcja analizuje wartość adresu i na tej podstawie deleguje operację do odpowiedniego komponentu emulatora.

Mapowanie przestrzeni adresowej jest opisana wcześniej w rozdziale 3.4

W przypadku trwającego transferu DMA dostęp do pamięci OAM jest blokowany, a magistrala zwraca wartość neutralną. Takie zachowanie odwzorowuje ograniczenia sprzętowe rzeczywistego systemu.

4.3.2 Operacje zapisu do magistrali

Procedura `bus_write(uint16_t address, uint8_t value)` realizuje zapis danych do przestrzeni adresowej. Mechanizm działania jest analogiczny do operacji odczytu — adres determinuje komponent docelowy operacji zapisu.

Zapis do pamięci ROM przekazywany jest do kontrolera kartridża, co umożliwia obsługę mechanizmów bankowania pamięci. Operacje zapisu do pamięci wideo oraz pamięci roboczej przekierowywane są do odpowiednich modułów pamięciowych. W obszarach zarezerwowanych lub nieużywanych operacja zapisu jest ignorowana.

Szczególnym przypadkiem jest zapis do adresu `0xFFFF`, który modyfikuje rejestr maski przerwań procesora. Mechanizm ten pozwala na dynamiczne sterowanie systemem przerwań w trakcie działania programu.

4.3.3 Operacje na słowach 16-bitowych

Magistrala udostępnia również funkcje operujące na danych 16-bitowych:

- `bus_read16(address)` — odczyt dwóch kolejnych bajtów i ich złożenie w słowo 16-bitowe,
- `bus_write16(address, value)` — zapis wartości 16-bitowej w postaci dwóch bajtów.

Operacje te wykorzystują konwencję zapisu w porządku little-endian, w której młodszy bajt przechowywany jest pod niższym adresem pamięci.

4.3.4 Znaczenie architektoniczne magistrali

el

Zastosowanie scentralizowanej warstwy magistrali zapewnia:

- jednoznaczne odwzorowanie przestrzeni adresowej systemu,
- izolację procesora od szczegółów implementacyjnych urządzeń peryferyjnych,
- możliwość łatwej rozbudowy emulatora o nowe komponenty,
- spójne modelowanie ograniczeń sprzętowych.

Magistrala stanowi zatem kluczowy element architektury emulatora, integrujący wszystkie moduły systemu w jednolitą strukturę komunikacyjną oraz zapewniający deterministyczne odwzorowanie zachowania sprzętu.

4.4 Implementacja pamięci operacyjnej RAM

Pamięć operacyjna systemu została odwzorowana w postaci struktury kontekstu zawierającej dwa fizycznie odrębne obszary pamięci: roboczą pamięć wewnętrzną WRAM oraz szybką pamięć wysoką HRAM. Oba segmenty reprezentowane są jako statyczne tablice bajtowe o stałym rozmiarze, co zapewnia deterministyczne odwzorowanie przestrzeni adresowej systemu.

Pamięć WRAM obejmuje zakres adresów od `0xC000` do `0xDFFF` i posiada rozmiar 8 KB. Z kolei HRAM zajmuje obszar od `0xFF80` do `0xFFFF` z efektywnym rozmiarem 128 B. Funkcja inicjalizacyjna ustawia wszystkie komórki obu obszarów na wartość początkową `0xFF`, co odpowiada stanowi pamięci po resecie systemu.

Dostęp do pamięci realizowany jest przez wyspecjalizowane funkcje odczytu i zapisu. Każda funkcja wykonuje translację adresu logicznego magistrali na indeks wewnętrznej tablicy poprzez odjęcie adresu bazowego segmentu (*Listing 2)*. W celu zapewnienia poprawności działania systemu implementacja zawiera kontrolę zakresu adresów. Próba dostępu poza dozwolony obszar skutkuje zgłoszeniem błędu i przerwaniem działania emulatora. Mechanizm ten zabezpiecza integralność modelu pamięci oraz ułatwia diagnostykę nieprawidłowych operacji.

```

struct ram_context {
    std::array<uint8_t, 0x2000> wram;
    std::array<uint8_t, 0x80> hram;
};

static ram_context ctx;

uint8_t wram_read(uint16_t address) {
    auto err = fmt::format("ERROR: Invalid wram read address 0x{:X}",
        address); // Wiadomosc o blednym adresie
```

```
    address -= 0xC000; // Odjecie adresu bazowego segmentu dla wram, dajac nam indeks tablicy ctx.wram
    if (address >= 0x2000) { // Jesli  indeks wychodzi poza zakres tablicy, dajemy komunikat o bledzie i konczymy prace programu
        fmt::println(err);
        exit(-1);
    }
    return ctx.wram.at(address);
}
```

Listing 2: Fragment implementacji pamięci operacyjnej

4.5 Implementacja stosu procesora

Stos procesora został zaimplementowany jako struktura danych działająca w przestrzeni pamięci systemowej i zarządzana przez rejestr wskaźnika stosu SP. Operacje na stosie wykorzystują interfejs magistrali systemowej, co zapewnia pełną zgodność z architekturą pamięci emulatora.

Operacja zapisu na stos realizowana jest poprzez dekrementację wskaźnika stosu, a następnie zapis wartości pod nowym adresem. Takie podejście odwzorowuje rzeczywiste działanie sprzętowego stosu, który rośnie w kierunku malejących adresów pamięci. Dla danych 16-bitowych stosowane są dwie kolejne operacje zapisu, rozpoczynające się od bajtu starszego, co zapewnia zgodność z reprezentacją little-endian.

Operacja odczytu ze stosu przebiega w odwrotnej kolejności. Najpierw odczytywana jest wartość spod adresu wskazywanego przez rejestr SP, a następnie wskaźnik stosu jest inkrementowany. W przypadku wartości 16-bitowych odczytywane są dwa kolejne bajty, które następnie łączone są w jedną wartość słowa maszynowego.

Takie podejście zapewnia pełną zgodność mechanizmu stosu z wymaganiami architektury procesora oraz umożliwia poprawną realizację instrukcji wywołań podprogramów, powrotów oraz obsługi przerwań.

4.6 Implementacja systemowego licznika czasowego

Odczyt instrukcji z tabeli opcodów w kontekście implementacji systemowego licznika czasowego powinien być zorganizowany w sposób przejrzysty i jednoznaczny. Każda instrukcja pobrana z tabeli opcodów odpowiada określonej liczbie cykli zegarowych, które bezpośrednio wpływają na aktualizację rejestru DIV. Po wykonaniu instrukcji emulator inkrementuje licznik DIV o liczbę cykli przypisaną do danej operacji.

Następnie, w oparciu o konfigurację rejestru TAC, analizowane są odpowiednie bity rejestru DIV w celu wykrycia opadającego zbocza sygnału zegarowego. W przypadku spełnienia warunku oraz aktywności licznika, następuje inkrementacja rejestru TIMA. Takie podejście zapewnia ścisłe powiązanie pomiędzy wykonywaniem instrukcji a mechanizmem odmierzania czasu.

Zaproponowany sposób integracji tabeli opcodów z systemem liczników czasowych umożliwia deterministyczne odwzorowanie zależności czasowych, gwarantując zgodność implementacji z semantyką sprzętową oraz poprawność działania emulatora.

4.7 Implementacja procesora

4.7.1 Reprezentacja i przechowywanie instrukcji

Najprostszym sposobem implementacji zestawu rozkazów procesora jest podejście naiwne, polegające na bezpośrednim powiązaniu każdego kodu operacji (*opcode*) z jego implementacją przy użyciu rozbudowanej instrukcji warunkowej lub konstrukcji `switch-case`.

Choć metoda ta jest koncepcyjnie prosta, prowadzi do istotnych problemów projektowych:

- silnego powiązania opisu instrukcji z jej implementacją,
- duplikacji logiki,
- ograniczonej skalowalności,
- utrudnionej rozbudowy i utrzymania kodu.

W celu eliminacji powyższych problemów zastosowano podejście rozdzielające:

1. opis semantyki instrukcji,
2. mechanizm dekodowania,
3. mechanizm wykonania.

Każda instrukcja reprezentowana jest przez strukturę danych zawierającą:

- typ operacji,
- tryb adresowania,
- rejestry źródłowe i docelowe,
- warunek wykonania,
- parametr dodatkowy.

```
    struct instruction {
        in_type type{IN_NONE}; // Typ operacji
        addr_mode mode{AM_IMP}; // Tryb adresowania
        reg_type reg1{RT_NONE}; // Rejestry zrodlowy
        reg_type reg2{RT_NONE}; // Rejestr docelowy
        cond_type cond{CT_NONE}; // warunek wykonania
        uint8_t param{0}; // Parametr dodatkowy
    };
```

Listing 3: Struktura instrukcji

Struktury te przechowywane są w tablicy indeksowanej bezpośrednio kodem operacji. Proces dekodowania sprowadza się do odczytu odpowiedniej struktury na podstawie wartości `opcode`, natomiast wykonanie delegowane jest do wyspecjalizowanej procedury przypisanej do typu instrukcji.

Takie rozwiązanie zapewnia wyraźną separację odpowiedzialności, upraszcza rozszerzanie zestawu rozkazów oraz ułatwia implementację mechanizmów debugowania i trybów adresowania.

```
constexpr instruction instructions[0x100] = {
    // 0x0X

    [0x00] = {.type = IN_NOP, .mode = AM_IMP},
    [0x01] = {.type = IN_LD, .mode = AM_R_D16, .reg1 = RT_BC},
    [0x02] = {.type = IN_LD, .mode = AM_MR_R, .reg1 = RT_BC, .reg2 = RT_A},
    [0x03] = {.type = IN_INC, .mode = AM_R, .reg1 = RT_BC},
    [0x04] = {.type = IN_INC, .mode = AM_R, .reg1 = RT_B},
    [0x05] = {.type = IN_DEC, .mode = AM_R, .reg1 = RT_B},
    [0x06] = {.type = IN_LD, .mode = AM_R_D8, .reg1=RT_B},

    // Pozostale instrukcje...
}
```

Listing 4: Fragment tablicy instrukcji indeksowanej kodem operacji

4.7.2 Struktura procesora i organizacja cyklu wykonawczego

Implementacja CPU opiera się na centralnej strukturze kontekstu wykonania, przechowującej pełny stan procesora oraz dane pomocnicze wykorzystywane podczas dekodowania i realizacji instrukcji.

```
    struct cpu_registers {
        uint8_t A;
        uint8_t F;
        uint8_t B;
        uint8_t C;
        uint8_t D;
        uint8_t E;
        uint8_t H;
        uint8_t L;
        uint16_t PC;
        uint16_t SP;
```

```
    };
```

Listing 5: Kontekst wykonania procesora

Stan architektoniczny przechowywany jest w strukturze `cpu_registers`, zawierającej:

Rejestry 8-bitowe:

- `A, F, B, C, D, E, H, L`

Rejestry sterujące:

- `PC` — licznik rozkazów (Program Counter),
- `SP` — wskaźnik stosu (Stack Pointer).

Rejestr `F` pełni funkcję rejestru flag i zawiera bity statusowe:

- Z — flaga zera,
- N — flaga odejmowania,
- H — flaga półprzeniesienia,
- C — flaga przeniesienia.

```
    struct cpu_context {
        cpu_registers regs;
        uint32_t ticks; // Liczniki taktow procesora

        uint16_t fetch_data; // Pobrany operand
        uint16_t mem_destination; // adres docelowy zapisu
        bool dest_is_mem; // flaga zapisu do pamirci
        uint8_t op_code; // aktualny kod operacji
        instruction const* cur_instruction;  //wskaznik na strukture
            instrukcji


        bool int_master_enabled; // Globalne zezwolenie na przerwanie
        bool enabling_ime; // Flaga aktywacji przerwan
        bool halted; //
        bool paused; // Stan zatrzymania procesora
        bool running; //

        uint8_t int_flags;
        uint8_t ie_register; // rejestr zgloszen
    };
```

Listing 6: Kontekst wykonania procesora

Pełny stan wykonawczy przechowywany jest w strukturze `cpu_context`, która oprócz rejestrów zawiera:

Dane operacyjne:

- `ticks` — licznik taktów procesora,
- `fetch_data` — pobrany operand,
- `mem_destination` — adres docelowy zapisu,
- `dest_is_mem` — flaga zapisu do pamięci,
- `op_code` — aktualny kod operacji,
- `cur_instruction` — wskaźnik na strukturę instrukcji.

Pola sterujące:

- globalne zezwolenie na przerwania,
- opóźnioną aktywację przerwań,
- stan zatrzymania procesora,

- rejestry zgłoszeń i masek przerwań.

4.7.3 Organizacja cyklu wykonawczego

Inicjalizacja Funkcja `cpu_init()` ustawia stan początkowy procesora: inicjalizuje rejestry, ustawia wartości `PC` i `SP`, konfiguruje system przerwań oraz inicjalizuje pamięć emulatora.

Pobieranie i dekodowanie instrukcji Funkcja `fetch_instruction()`:

1. odczytuje kod operacji z pamięci pod adresem wskazywanym przez `PC`,
2. inkrementuje licznik rozkazów,
3. wyszukuje strukturę opisującą instrukcję.

Pobieranie operandów Funkcja `fetch_data()` interpretuje tryb adresowania i przygotowuje dane dla etapu wykonania. Operandy mogą pochodzić z:

- rejestrów procesora,
- pamięci operacyjnej,
- danych natychmiastowych zapisanych w kodzie programu.

Obsługiwane są także tryby autoinkrementacji i autodekrementacji.

Wykonanie instrukcji Funkcja `execute()` wybiera procedurę wykonawczą na podstawie typu operacji i przekazuje do niej kontekst procesora. Wykorzystanie tablicy wskaźników funkcji umożliwia szybkie mapowanie typu instrukcji na jej implementację.

Klasy procedur wykonawczych **Operacje arytmetyczne i logiczne:**

- dodawanie, odejmowanie,
- operacje bitowe,
- porównania,
- aktualizacja flag statusowych.

Operacje przepływu sterowania:

- skoki warunkowe i bezwarunkowe,
- wywołania i powroty,
- operacje na stosie.

Instrukcje bitowe i rotacyjne:

- manipulacja pojedynczymi bitami,
- przesunięcia,
- rotacje.

Instrukcje systemowe:

- zarządzanie przerwaniami,
- zatrzymanie procesora,
- operacje specjalne.

Symulacja cykli zegarowych Funkcja `emu_cycles()` odpowiada za synchronizację systemu. Każdy cykl powoduje:

- inkrementację `ticks`,
- aktualizację timera,
- aktualizację układu graficznego,
- obsługę kontrolera DMA.

Główny krok procesora Funkcja `cpu_step()` realizuje pojedynczy krok pracy CPU:

1. pobranie instrukcji,
2. symulację cyklu zegarowego,
3. pobranie operandów,
4. wykonanie instrukcji,
5. obsługę przerwań,
6. aktualizację transferów DMA.

W stanie zatrzymania wykonywane są wyłącznie cykle zegarowe do momentu zgłoszenia przerwania.

4.7.4 Warstwa dostępu do rejestrów

Wprowadzono ujednoliconą warstwę dostępu do rejestrów 8- i 16-bitowych.

Odczyt rejestrów Funkcja `cpu_read_reg(reg_type rt)`:

- dla rejestrów 8-bitowych wykonuje bezpośredni odczyt,
- dla rejestrów złożonych (AF, BC, DE, HL) interpretuje pary rejestrów jako wartości 16-bitowe.

Zapis rejestrów Funkcja `cpu_set_reg(reg_type rt, uint16_t value)` umożliwia zapis wartości do wskazanego rejestru z zachowaniem spójności reprezentacji.

Dodatkowe funkcje `cpu_read_reg8` oraz `cpu_set_reg8` zapewniają bezpośredni dostęp do rejestrów bajtowych.

Zarządzanie flagami Funkcja `cpu_set_flags` umożliwia selektywną modyfikację flag statusowych oraz maskowanie niedozwolonych bitów rejestru `F`.

4.7.5 Wykonanie w środowisku wielowątkowym

Funkcja `cpu_run_threaded()` realizuje pracę procesora w dedykowanym wątku. Odpowiada za:

- inicjalizację komponentów,
- ciągłe wykonywanie `cpu_step()`,
- zbieranie statystyk wydajności,
- synchronizację z modułem graficznym.

Oddzielenie cyklu CPU do dedykowanego wątku pozwala zachować stabilną prędkość emulacji oraz umożliwia równoległą obsługę warstwy prezentacji i interfejsu użytkownika.

4.8 Implementacja przetwarzania grafiki

Moduł przetwarzania grafiki odpowiada za emulację działania kontrolera LCD konsoli Game Boy. Jego zadaniem jest odwzorowanie rejestrów sterujących układu graficznego, zarządzanie paletami kolorów oraz obsługa komunikacji z magistralą pamięci emulatora. Implementacja została zaprojektowana w sposób modularny, tak aby możliwe było jednoznaczne odwzorowanie stanu sprzętowego w strukturze kontekstu oraz zapewnienie spójnego interfejsu dostępu do rejestrów.

4.8.1 Reprezentacja rejestrów LCD i obsługa palet

Stan kontrolera LCD przechowywany jest w strukturze `lcd_context`. Struktura ta stanowi programowe odwzorowanie zestawu rejestrów sprzętowych oraz danych pomocniczych wykorzystywanych podczas procesu renderowania obrazu. W szczególności zawiera ona:

- rejestry kontrolne `lcdc` oraz `lcds`,
- rejestry przewijania obrazu `scroll_x` oraz `scroll_y`,

- licznik aktualnie renderowanej linii `ly` oraz rejestr porównania `ly_compare`,
- rejestr odpowiedzialny za transfer DMA,
- rejestry palet kolorów dla tła oraz sprite'ów,
- współrzędne okna renderowania,
- tablice kolorów wykorzystywane podczas rasteryzacji.

Zastosowanie pojedynczej struktury kontekstu umożliwia bezpośrednie odwzorowanie przestrzeni adresowej rejestrów LCD w pamięci programu. Funkcja `lcd_get_context()` zwraca wskaźnik do globalnej instancji tej struktury, co upraszcza dostęp do aktualnego stanu układu graficznego w pozostałych modułach emulatora.

Makra dostępu do bitów sterujących W celu uproszczenia interpretacji bitów rejestrów sterujących zdefiniowano zestaw makr umożliwiających wykonywanie najczęściej wykorzystywanych operacji logicznych. Makra te pozwalają między innymi na:

- odczyt flag kontrolnych rejestru `lcdc`,
- wybór odpowiednich obszarów pamięci wykorzystywanych podczas renderowania,
- ustawianie oraz odczyt aktualnego trybu pracy kontrolera LCD,
- obsługę źródeł przerwań rejestru statusowego.

Zastosowanie warstwy abstrakcji w postaci makr poprawia czytelność kodu oraz ogranicza ryzyko błędów wynikających z bezpośredniej manipulacji maskami bitowymi.

Inicjalizacja kontekstu LCD Funkcja `lcd_init()` odpowiada za inicjalizację kontekstu kontrolera LCD. W trakcie jej wykonywania ustawiany jest początkowy stan rejestrów oraz domyślne wartości palet kolorów. W implementacji wykorzystano tablicę `colors_defaults`, zawierającą cztery podstawowe kolory wykorzystywane podczas renderowania obrazu.

Podczas inicjalizacji wykonywane są następujące operacje:

- wyzerowanie rejestrów sterujących,
- ustawienie domyślnych wartości rejestrów palet,
- inicjalizacja tablic kolorów tła oraz obiektów.

Aktualizacja palet kolorów Dekodowanie zawartości rejestrów palet realizowane jest przez funkcję `update_pallete()`. Zadaniem tej funkcji jest przekształcenie wartości zapisanej w rejestrze palety w zestaw czterech rzeczywistych wartości kolorów wykorzystywanych podczas generowania pikseli.

Każde dwa bity rejestru palety reprezentują indeks koloru w tablicy domyślnej. Funkcja:

- wybiera odpowiednią tablicę kolorów (tła lub sprite'ów),
- dekoduje kolejne pary bitów,
- przypisuje im odpowiadające wartości 32-bitowych kolorów.

Oddzielenie reprezentacji rejestru palety od rzeczywistych wartości kolorów eliminuje konieczność wykonywania operacji dekodowania podczas zapisu każdego piksela, co pozytywnie wpływa na wydajność renderowania.

Obsługa odczytu i zapisu rejestrów Dostęp do rejestrów LCD realizowany jest poprzez funkcje `lcd_read()` oraz `lcd_write()`, które odwzorowują pamięciowe mapowanie rejestrów kontrolera graficznego.

Funkcja `lcd_read()` oblicza przesunięcie względem bazowego adresu przestrzeni rejestrów i zwraca odpowiednią wartość z kontekstu LCD.

Z kolei funkcja `lcd_write()` obsługuje zapis do rejestrów, uwzględniając przy tym szczególne przypadki, takie jak uruchomienie transferu DMA lub aktualizacja rejestrów palet kolorów. Po wykryciu zapisu do rejestru palety funkcja wywołuje mechanizm aktualizacji tablic kolorów wykorzystywanych podczas rasteryzacji.

4.8.2 Renderowanie tła oraz obiektów OAM

Proces generowania obrazu realizowany jest w trybie transferu pikseli (`MODE_XFER`) i opiera się na mechanizmie potoku przetwarzania współpracującego z kolejką FIFO pikseli. Implementacja odwzorowuje sprzętowy model działania kontrolera graficznego, w którym dane tła oraz sprite'ów są pobierane, dekodowane i łączone przed zapisaniem do bufora ramki.

Renderowanie tła Renderowanie tła realizowane jest przez strukturę `pixel_fifo_context` oraz funkcję `pipeline_process()`, wywoływaną cyklicznie podczas pracy trybu transferu pikseli.

Dla każdej linii obrazu wyznaczane są następujące parametry:

- współrzędne mapy tła (`map_x`, `map_y`),
- przesunięcie w obrębie kafelka (`tile_y`),
- aktualna pozycja piksela w linii obrazu.

Pobieranie danych z pamięci VRAM realizowane jest przy użyciu maszyny stanów `fetch_state`, obejmującej następujące etapy:

1. `FS_TILE` — odczyt indeksu kafelka z mapy tła,
2. `FS_DATA0` — odczyt młodszego bajtu opisującego linię pikseli kafelka,
3. `FS_DATA1` — odczyt starszego bajtu linii pikseli,
4. `FS_PUSH` — dekodowanie ośmiu pikseli i zapis ich do kolejki FIFO.

Każdy kafelek reprezentowany jest przez dwa bajty opisujące jedną linię pikseli. Dla każdego z ośmiu pikseli wyznaczana jest dwubitowa wartość koloru poprzez ekstrakcję odpowiednich bitów z obu bajtów. Wartość ta mapowana jest następnie na rzeczywisty kolor przy użyciu aktualnej palety tła.

Wygenerowane piksele zapisywane są do kolejki FIFO przy użyciu funkcji `pixel_fifo_push()`. Następnie funkcja `pipeline_push_pixel()` usuwa piksele z kolejki i zapisuje je do bufora ramki, uwzględniając:

- poziome przewijanie obrazu (`scroll_x`),
- numer aktualnej linii (`ly`),
- bieżącą pozycję piksela w linii.

Takie podejście umożliwia przyrostowe generowanie obrazu zgodnie z kolejnymi cyklami pracy jednostki PPU.

Renderowanie obiektów OAM Renderowanie sprite'ów realizowane jest równolegle z generowaniem warstwy tła i polega na odpowiednim łączeniu obu warstw z uwzględnieniem priorytetów.

Selekcja sprite'ów dla linii W trybie `MODE_OAM` wykonywana jest funkcja `load_line_sprites()`, której zadaniem jest:

- przeszukanie pamięci OAM,
- wybór sprite'ów przecinających aktualną linię obrazu,
- ograniczenie ich liczby do maksymalnie dziesięciu obiektów,
- posortowanie ich według współrzędnej poziomej.

Wybrane sprite'y zapisywane są w strukturze pomocniczej `line_sprites`, wykorzystywanej podczas dalszego procesu renderowania.

Łączenie warstw Podczas generowania pikseli wykonywana jest funkcja `fetch_sprite_pixels()`, która sprawdza, czy sprite pokrywa aktualnie renderowaną pozycję X. Jeżeli tak, wykonywane są następujące operacje:

- uwzględnienie odbicia poziomego i pionowego,
- dekodowanie wartości koloru sprite'a,

- sprawdzenie przezroczystości piksela,
- porównanie priorytetu względem tła.

Jeżeli warunki priorytetu są spełnione, kolor piksela tła zostaje zastąpiony kolorem sprite'a wybranym z odpowiedniej palety obiektów.

4.8.3 Maszyna stanów kontrolera LCD

Przetwarzanie obrazu realizowane jest w postaci deterministycznej maszyny stanów zsynchronizowanej z licznikiem cykli. Aktualny tryb pracy kontrolera przechowywany jest w dwóch najmłodszych bitach rejestru statusowego i może przyjmować jedną z czterech wartości: `MODE_OAM`, `MODE_XFER`, `MODE_HBLANK` oraz `MODE_VBLANK`.

Tryb OAM W trybie OAM analizowana jest pamięć obiektów w celu wyznaczenia sprite'ów widocznych w bieżącej linii obrazu.

Tryb transferu pikseli W trybie `MODE_XFER` realizowany jest właściwy proces generowania pikseli, obejmujący pobieranie danych kafelków, dekodowanie pikseli oraz łączenie warstw tła i sprite'ów.

Tryb HBLANK Tryb `MODE_HBLANK` stanowi okres wygaszania poziomego, w którym nie są generowane nowe piksele, a procesor uzyskuje dostęp do pamięci wideo.

Tryb VBLANK Tryb `MODE_VBLANK` obejmuje linie poza obszarem widocznym. W tej fazie generowane jest przerwanie synchronizacji pionowej oraz możliwa jest aktualizacja bufora ramki.

Synchronizacja z rejestrem LYC Dodatkowym elementem działania kontrolera jest mechanizm porównania rejestru `LY` z rejestrem `LYC`. W przypadku zgodności ustawiana jest flaga statusowa, która może generować przerwanie, jeżeli odpowiednie źródło zostało wcześniej włączone.

Podsumowanie Zaproponowana architektura renderowania oparta na maszynie stanów oraz potoku przetwarzania pikseli umożliwia wierne odwzorowanie działania sprzętowego kontrolera LCD. Jednocześnie zachowana została modularność implementacji, co ułatwia dalszy rozwój emulatora oraz analizę jego działania.

Zastosowanie jawnej maszyny stanów:

- zapewnia zgodność czasową z przebiegiem generowania linii,
- rozdziela logicznie poszczególne etapy renderowania,
- upraszcza obsługę przerwań i synchronizacji z procesorem,
- umożliwia deterministyczne odwzorowanie przebiegu całej ramki.

4.9 Implementacja interfejsu graficznego emulatora

Interfejs użytkownika emulatora został zaimplementowany w postaci klasy `RayboyUI`. Klasa ta odpowiada za wizualizację działania emulatora, obsługę wejścia użytkownika oraz prezentację informacji diagnostycznych dotyczących pracy systemu emulacji. Warstwa graficzna została zrealizowana z wykorzystaniem biblioteki raylib, która umożliwia tworzenie aplikacji graficznych 2D oraz obsługę urządzeń wejścia takich jak klawiatura.

4.9.1 Opis klasy

`RayboyUI` zarządza interfejsem graficznym emulatora. Odpowiada za pętlę renderowania, obsługę okna oraz synchronizację z wątkiem CPU za pośrednictwem współdzielonego stanu (`EmulatorShared`).

4.9.2 Interfejs publiczny

Metoda	Opis
`setup(EmulatorShared*)`	Inicjalizuje UI; przyjmuje wskaźnik na współdzielony stan
`shouldExit()`	Zwraca `true`, gdy użytkownik zamknął okno
`update()`	Obsługuje eventy wejściowe i logikę UI
`draw()`	Renderuje aktualną klatkę
`shutdown()`	Zwalnia zasoby UI

4.9.3 Przykład działania korzystania z klasy RayboyUI

```cpp
#include "RayboyUI.h"
#include "EmulatorShared.h"
#include <thread>

// Globalny wskaznik -- UI i watek CPU komunikuja sie przez EmulatorShared
EmulatorShared* g_shared;

int main(int argc, char** argv) {
    // 1. Zaladuj ROM (warunek wstepny)
    if (argc < 2 || !cart_load(argv[1]))
    return 1;

    // 2. Utworz wspoldzielony stan i UI
    EmulatorShared shared;
    RayboyUI ui;
    g_shared = &shared;

    // 3. Zainicjalizuj UI ze wspoldzielonym stanem
    ui.setup(g_shared);

    // 4. Uruchom watek CPU w tle
    std::thread cpu_thread(cpu_run_threaded, g_shared);

    // 5. Petla glowna -- UI zyje w watku glownym
    while (!ui.shouldExit()) {
        ui.update(); // eventy, input
        ui.draw();   // renderowanie
    }

    // 6. Sygnalizuj watkowi CPU zakonczenie i poczekaj na niego
    shared.should_exit.store(true, std::memory_order_release);
    if (cpu_thread.joinable())
    cpu_thread.join();

    // 7. Wyczysc zasoby UI
    g_shared = nullptr;
    ui.shutdown();

    return 0;
}
```

Listing 7: Minimalny przykład użycia klasy `RayboyUI`

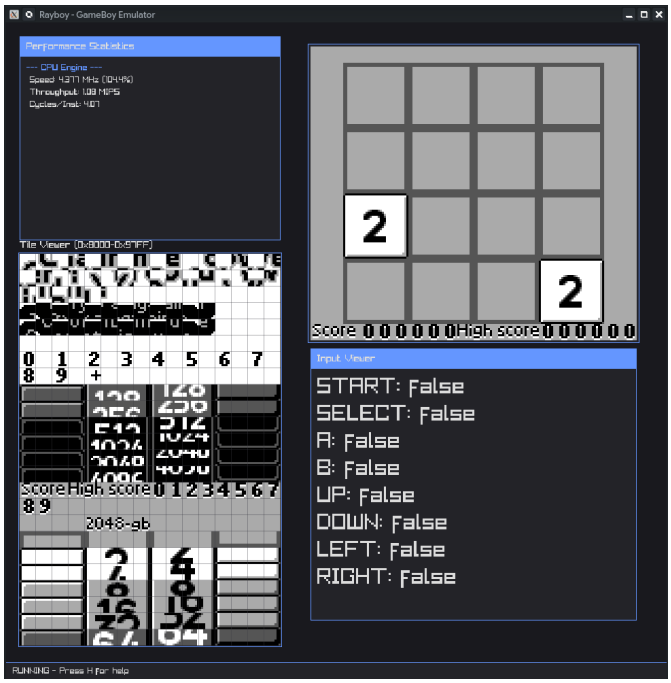
4.9.4 Kluczowe założenia projektowe

UI w wątku głównym. Klasa `RayboyUI` musi działać w wątku głównym — jest to wymaganie większości bibliotek graficznych (np. raylib, SDL).

CPU w osobnym wątku. Emulacja procesora przebiega równolegle; komunikacja z UI odbywa się przez strukturę `EmulatorShared` z atomowym polem `should_exit`.

Kolejność zamykania. Prawidłowa sekwencja shutdown to: sygnał do wątku CPU → `join()` → `shutdown()` UI. Zapobiega to sytuacji, w której wątek CPU odwołuje się do zwolnionego wskaźnika `g_shared`.

4.9.5 Struktura klasy



Rysunek 5: Okno emulatora. Źródło: Własnoręczna implementacja

Klasa `RayboyUI` zarządza wszystkimi elementami graficznymi interfejsu użytkownika. Przechowuje ona wskaźnik do struktury współdzielonej `EmulatorShared`, która umożliwia wymianę danych pomiędzy rdzeniem emulatora a warstwą graficzną. Struktura ta zawiera między innymi:

- bufor klatek obrazu generowanych przez emulator,
- dane graficzne kafelków zapisanych w pamięci wideo,
- statystyki pracy procesora emulowanego systemu,
- flagi sterujące stanem emulatora (np. pauza lub zakończenie pracy).

Klasa przechowuje również zmienne opisujące aktualny stan interfejsu, takie jak widoczność poszczególnych paneli, w tym podglądu kafelków graficznych, panelu statystyk oraz ekranu pomocy.

Dodatkowo zdefiniowane zostały stałe określające parametry wyświetlania, takie jak rozdzielczość ekranu oryginalnej konsoli Game Boy (160×144 piksele), współczynnik skalowania obrazu oraz liczba wyświetlanych kafelków w podglądzie pamięci graficznej.

4.9.6 Główna pętla interfejsu

Działanie interfejsu użytkownika opiera się na dwóch podstawowych metodach: `update()` oraz `draw()`.

Metoda `update()` odpowiada za aktualizację stanu interfejsu, w tym obsługę wejścia użytkownika oraz aktualizację danych graficznych. Natomiast metoda `draw()` realizuje renderowanie wszystkich elementów interfejsu, takich jak ekran emulatora, panel statystyk, podgląd kafelków oraz pasek statusu.

4.9.7 Inicjalizacja interfejsu

Proces inicjalizacji interfejsu realizowany jest przez metodę `setup()`. W trakcie jej wykonywania obliczane są wymiary okna aplikacji na podstawie rozmiarów poszczególnych paneli interfejsu. Następnie wyznaczane są pozycje elementów graficznych, takich jak panel statystyk, podgląd kafelków oraz ekran emulatora.

Kolejnym krokiem jest utworzenie okna aplikacji oraz ustawienie docelowej liczby klatek na sekundę. Następnie tworzone są struktury danych typu `Image` oraz `Texture2D`, które przechowują

obraz ekranu emulatora oraz dane graficzne podglądu kafelków. Obrazy te są następnie przesyłane do pamięci karty graficznej w postaci tekstur, co umożliwia ich szybkie renderowanie.

4.9.8 Aktualizacja obrazu ekranu

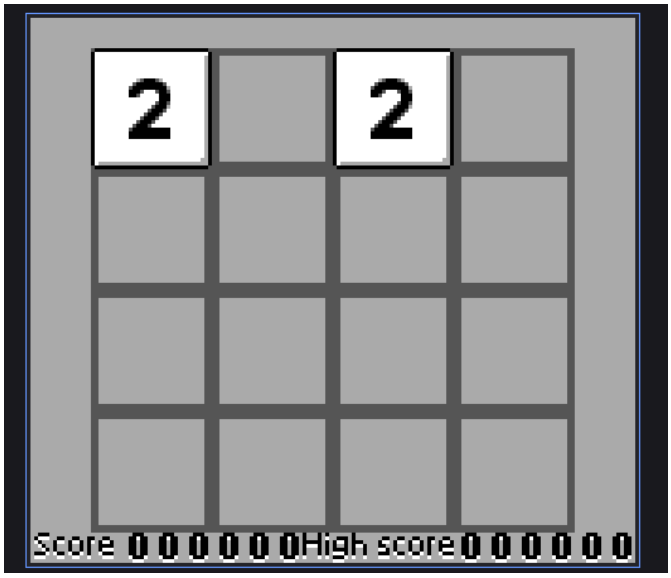
Aktualizacja obrazu wyświetlanego przez emulator realizowana jest w metodzie `updateScreenTexture()`. W pierwszej kolejności sprawdzana jest flaga `frame_ready`, która informuje o dostępności nowej klatki obrazu wygenerowanej przez rdzeń emulatora.

W przypadku dostępności nowej klatki wykonywane są następujące operacje:

1. odczyt indeksu bufora zawierającego aktualną klatkę obrazu,
2. skopiowanie danych pikseli do lokalnego obrazu interfejsu,
3. aktualizacja tekstury znajdującej się w pamięci karty graficznej,
4. wyzerowanie flagi informującej o gotowości klatki.

Zastosowanie takiego mechanizmu pozwala na bezpieczną komunikację pomiędzy wątkiem emulatora a wątkiem odpowiedzialnym za renderowanie interfejsu użytkownika.

4.9.9 Renderowanie obrazu emulatora

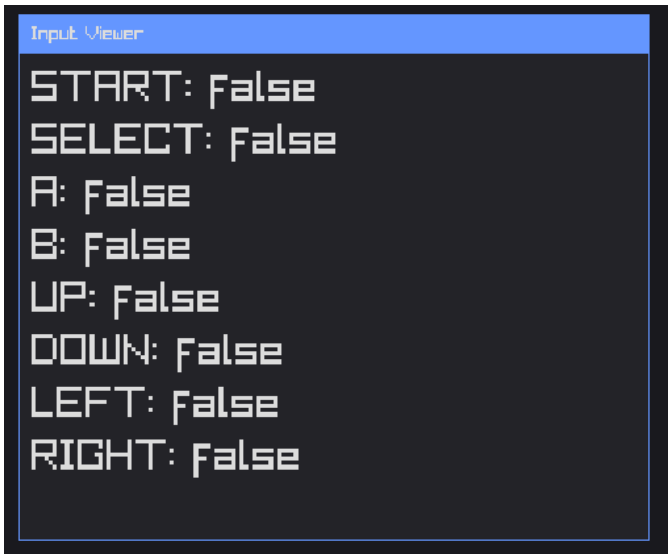


Rysunek 6: Zrzut emulowanego ekranu. Źródło: Własnoręczna implementacja

Wyświetlanie obrazu generowanego przez emulator realizowane jest w metodzie `drawScreen()`. W pierwszej kolejności rysowana jest ramka otaczająca ekran emulatora. Następnie wyświetlana jest tekstura zawierająca aktualną klatkę obrazu.

Ponieważ rozdzielczość oryginalnej konsoli Game Boy jest stosunkowo niewielka, obraz jest skalowany podczas renderowania. Dzięki temu możliwe jest jego czytelne wyświetlenie na ekranie komputera

4.9.10 Obsługa wejścia użytkownika



Rysunek 7: Okienko stanu wirtualnych urządeń wejścia konsoli. Źródło: Własnoręczna implementacja

Obsługa wejścia z klawiatury realizowana jest w metodzie `handleInput()`. W metodzie tej zdefiniowano zestaw skrótów klawiszowych umożliwiających sterowanie pracą emulatora.

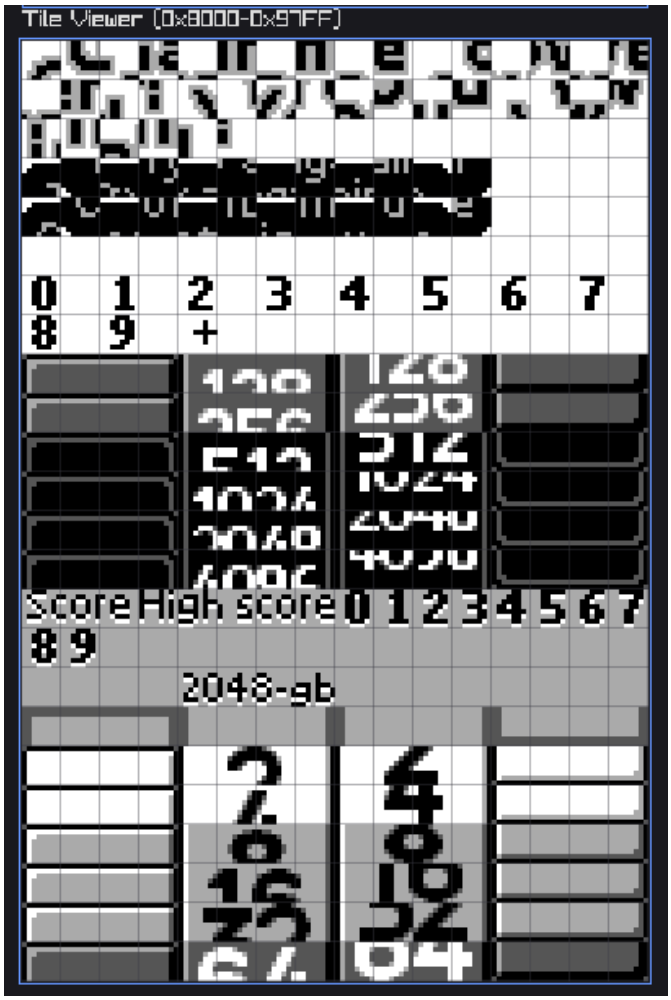
Do najważniejszych funkcji należą:

- wstrzymanie lub wznowienie emulacji,
- zakończenie działania programu,

- włączenie lub wyłączenie panelu statystyk,
- włączenie lub wyłączenie podglądu kafelków,
- wyświetlenie ekranu pomocy.

Dodatkowo klawisze klawiatury mapowane są na przyciski kontrolera konsoli Game Boy. Stan przycisków zapisywany jest w strukturze `gamepad_state`, która następnie przekazywana jest do rdzenia emulatora.

4.9.11 Podgląd kafelków graficznych



Rysunek 8: Podgląd kafelków przechowywanych w pamięci VRAM. Źródło: Własnoręczna implementacja

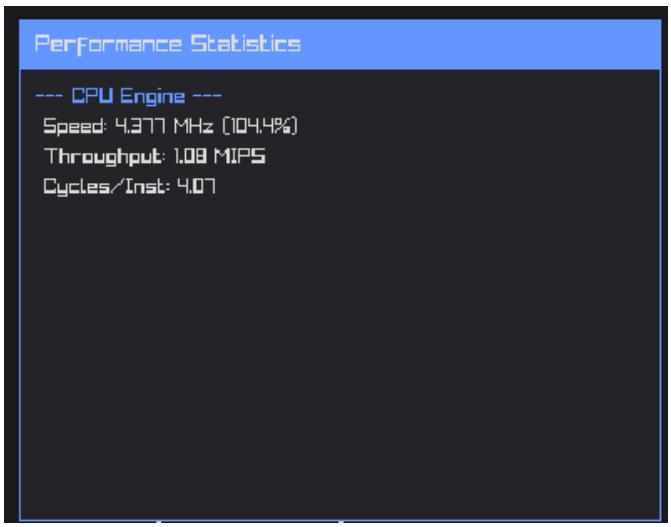
Konsola Game Boy wykorzystuje system grafiki oparty na tzw. kafelkach (*tiles*), które są niewielkimi obrazami o rozmiarze 8×8 pikseli. Metoda `updateTileImage()` odpowiada za generowanie obrazu zawierającego podgląd wszystkich kafelków zapisanych w pamięci graficznej emulatora.

Dla każdego kafelka wywoływana jest funkcja `renderTile()`, która odczytuje dane pikseli z pamięci wideo, oblicza identyfikator koloru dla poszczególnych pikseli oraz rysuje je w odpowiednim miejscu obrazu podglądu.

Podgląd ten jest następnie wyświetlany w interfejsie przy użyciu metody `drawTileViewer()`, która dodatkowo rysuje siatkę ułatwiającą identyfikację poszczególnych kafelków.

4.9.12 Panel statystyk

Interfejs zawiera również panel statystyk wydajności emulatora, który wyświetlany jest przez metodę `drawStatsPanel()`. Panel ten prezentuje informacje dotyczące pracy rdzenia emulatora, takie jak:

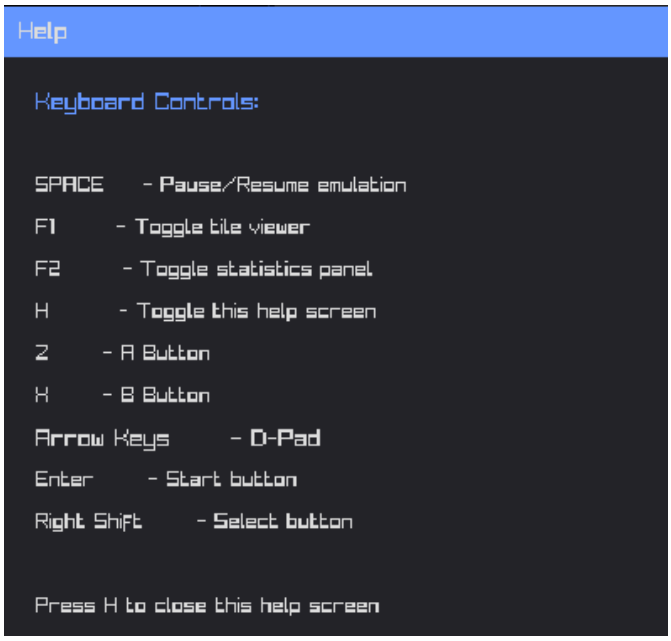


Rysunek 9: Okienko statystyk emulacji. Źródło: Własnoręczna implementacja

- aktualna prędkość emulowanego procesora wyrażona w MHz,
- procentowa zgodność prędkości z oryginalnym sprzętem,
- liczba wykonywanych instrukcji na sekundę (MIPS),
- średnia liczba cykli procesora przypadających na jedną instrukcję.

Informacje te są pomocne podczas analizy wydajności implementacji emulatora oraz w procesie jego optymalizacji.

4.9.13 Ekran Pomocy



Rysunek 10: Ekran pomocy. Źródło: Własnoręczna implementacja

4.9.14 Podsumowanie

Klasa `RayboyUI` stanowi warstwę prezentacji emulatora i pełni rolę pośrednika pomiędzy użytkownikiem a rdzeniem systemu emulacji. Odpowiada ona zarówno za wizualizację obrazu generowanego przez emulator, jak i za obsługę wejścia użytkownika oraz prezentację informacji diagnostycznych.

Zastosowanie biblioteki raylib pozwoliło na stosunkowo prostą implementację graficznego interfejsu użytkownika przy jednoczesnym zachowaniu dobrej wydajności renderowania.

4.10 Weryfikacja implementacji emulatora

Weryfikacja implementacji emulatora ma na celu potwierdzenie poprawności odwzorowania zachowania oryginalnego sprzętu. Ze względu na złożoność architektury konsoli Game Boy oraz dużą liczbę zależności pomiędzy jej komponentami, nawet niewielkie odstępstwa od specyfikacji mogą prowadzić do niepoprawnego działania uruchamianych programów. W procesie weryfikacji zastosowano zestaw testów diagnostycznych oraz porównania z referencyjnymi implementacjami emulatorów.

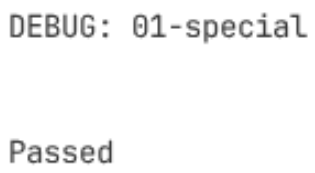
4.10.1 Metodyka weryfikacji

Proces weryfikacji opiera się na trzech komplementarnych podejściach. Pierwszym z nich są automatyczne testy diagnostyczne sprawdzające poprawność implementacji podstawowych elementów architektury, takich jak zestaw instrukcji procesora, obsługa przerwań oraz działanie timerów. Drugim etapem jest analiza szczegółowych logów wykonywania programu w przypadku wykrycia niezgodności w działaniu emulatora. Trzecim elementem weryfikacji jest uruchamianie rzeczywistych programów przeznaczonych dla platformy Game Boy oraz porównanie ich działania z referencyjnym emulatorem.

Takie podejście pozwala na weryfikację zarówno niskopoziomowych mechanizmów sprzętowych, jak i poprawności działania emulatora z punktu widzenia użytkownika końcowego.

4.10.2 Narzędzia testowe

Podstawowym zestawem testów wykorzystanym w procesie weryfikacji były testy autorstwa *blargg*[5]. Zestaw ten jest szeroko stosowany w społeczności twórców emulatorów i służy do sprawdzania poprawności implementacji procesora oraz wybranych elementów sprzętowych konsoli. Testy te weryfikują między innymi działanie instrukcji procesora, obsługę przerwań oraz funkcjonowanie timerów.



Rysunek 11: Wynik przechodzącego poprawnie testu *01-special.gb*. Własnoręczny zrzut ekranu

W przypadku niepowodzenia testów przeprowadzana jest dodatkowa analiza przy użyciu narzędzia *gameboy-doctor*[7]. Emulator generuje plik zawierający szczegółowe logi wykonywania programu, obejmujące stan procesora dla każdego cyklu pracy, w tym wartości rejestrów oraz aktualnie wykonywaną instrukcję.

Wygenerowany plik jest następnie przekazywany do narzędzia *gameboy-doctor*, które porównuje go z referencyjnymi logami uzyskanymi z poprawnie działającego emulatora. Narzędzie identyfikuje pierwszy moment, w którym stan procesora zaczyna odbiegać od wartości oczekiwanych, co umożliwia precyzyjne zlokalizowanie błędu w implementacji.

4.10.3 Testy porównawcze

Ostatnim etapem weryfikacji było porównanie działania emulatora z dojrzałą implementacją referencyjną. W tym celu wykorzystano emulator *SameBoy*[6], który jest uznawany za jedną z najbardziej dokładnych implementacji emulatora konsoli Game Boy.

```
A:00 F:00 B:95 C:60 D:72 E:75 H:C3 L:37 SP:DFFF PC:C7D2 PCMEM:18,FE,C9,00
DEBUG: 01-special

00000000
DAA

Failed #6
```

Rysunek 12: Zrzut terminalu z źle zaimplementowanej instrukcji. Własnoręczny zrzut ekranu

```
mrdeun on Fedora via ⎇ HEAD at …/build/clang took ⌛ 35s
~/Workspace/gameboy-doctor/gameboy-doctor ./logs.txt cpu_instrs 1
============== ERROR ==============

Mismatch in CPU state at line 32529:

MINE:   A:31 F:-HN- B:0B C:FF D:C7 E:9B H:FF L:83 SP:DFE9 PC:C062 PCMEM:2D,4E,2D,56
YOURS:  A:31 F:-HN- B:FF C:FF D:C7 E:9B H:FF L:83 SP:DFE9 PC:C062 PCMEM:2D,4E,2D,56

The CPU state before this (at line 32528) was:

        A:31 F:60 B:00 C:FF D:C7 E:9B H:FF L:83 SP:DFE9 PC:C061 PCMEM:46,2D,4E,2D

The last operation executed (in between lines 32528 and 32529) was:

        0x46 LD B (HL)

Perhaps the problem is with this opcode, or with your interrupt handling?
```

Rysunek 13: Przykład diagnozy w którym gameboy-doctor wykazał niezgodność procesora. Własnoręczny zrzut ekranu

Porównanie polegało na uruchomieniu tej samej aplikacji typu homebrew w obu emulatorach. Jako program testowy wykorzystano implementację gry *2048* [12]. Analizie poddano poprawność działania logiki gry, sposób renderowania grafiki oraz reakcję programu na dane wejściowe użytkownika. Zgodność zachowania aplikacji w obu emulatorach stanowi dodatkowe potwierdzenie poprawności implementacji.

4.11 Problemy implementacyjne

Przedstawiona implementacja tablicy instrukcji wykorzystuje dwie niestandardowe konstrukcje językowe, które nie są powszechnie wspierane przez kompilatory C++.

Pierwszą z nich są *designated initializers* (oznaczone inicjalizatory pól struktury), wprowadzone oficjalnie w standardzie C++20 [8]:

```
vec2 v = {.x = 12.0, .y = 0.3};
```

Listing 8: Oznaczone inicjalizatory pól struktury

Drugą konstrukcją są *designated initializers* dla tablic, zaczerpnięte z języka C99, lecz nieuwzględnione w żadnym standardzie C++:

```
vec2 res[5] = {
    [2] = {.x = 12.0, .y = 0.3},
};
```

Listing 9: Nieoznaczone inicjalizatory tablic – rozszerzenie niestandardowe

O ile oznaczone inicjalizatory pól są obsługiwane przez większość nowoczesnych kompilatorów zgodnych z C++20, o tyle inicjalizatory indeksowane dla tablic stanowią rozszerzenie niestandardowe. Aktualnie jedynym kompilatorem C++ obsługującym tę składnię jest `clang++`. Kompilatory `g++` oraz `MSVC` odrzucają ten kod nawet przy włączonym standardzie C++23.

Konsekwencją powyższego jest ograniczona przenośność kodu – projekt można zbudować wyłącznie przy użyciu `clang++`, co należy jawnie zaznaczyć w systemie budowania, na przykład wymuszając wybór kompilatora w pliku `CMakeLists.txt`:

```
set(CMAKE_CXX_COMPILER "clang++" CACHE STRING "" FORCE)
```

Listing 10: Wymuszenie kompilatora `clang++` w CMake

5 Działanie programu

5.1 Struktura Programu

Kompilacja projektu zwraca plik wykonywalny który należy uruchomić z poziomu terminalu - to uruchomienia pliku gry w emulatorze, należy dać jako argument ścieżke do ROMu *np. ./Rayboy ./2048.gb*.

5.2 Sterowanie w trakcji działania emulatora

- Kontrola konsoli
  - Góra - Strzałka w góre
  - Dół - Strzałka w dół
  - Lewo - Strzałka w lewo
  - Prawo - Strzałka w prawo
  - START - Enter
  - SELECT - Prawy Shift
  - B - B
  - A - A
- Kontrola interfejsu emulatora
  - Wszytrzymanie/Wznowienie emulacji - Spacja
  - Przełączenie widoku na kafelki - F1
  - Przełączenie widoku statystyk emulatora - F2
  - Przełączenie ekranu pomocy - H

6 Zakończenie

Celem niniejszej pracy było zaprojektowanie oraz implementacja emulatora konsoli Game Boy. Realizacja tego zadania pozwoliła na praktyczne poznanie wielu zagadnień związanych z architekturą systemów komputerowych, w szczególności z działaniem procesorów oraz sposobem wykonywania instrukcji na niskim poziomie abstrakcji. Implementacja jednostki centralnej emulatora wymagała dokładnego zrozumienia zestawu instrukcji procesora oraz mechanizmu ich interpretacji i wykonywania w środowisku programowym.

Istotnym elementem projektu było również odtworzenie sposobu generowania obrazu przez oryginalny sprzęt. Implementacja części graficznej pozwoliła lepiej zrozumieć, w jaki sposób dane graficzne są przechowywane w pamięci oraz jak są interpretowane przez układ graficzny w celu wygenerowania obrazu na ekranie. Proces ten wymagał odwzorowania specyficznych mechanizmów działania sprzętu oraz odpowiedniej synchronizacji pomiędzy komponentami emulatora.

W trakcie realizacji projektu szczególnie istotną rolę odegrało testowanie poprawności działania emulatora. Ze względu na złożoność emulowanego systemu konieczne było porównywanie zachowania implementacji z rezultatami uzyskiwanymi przez sprawdzone i poprawnie działające emulatory. Tego rodzaju weryfikacja pozwalała na identyfikację błędów oraz stopniowe zwiększanie zgodności implementacji z zachowaniem oryginalnego sprzętu.

Obecna wersja emulatora obsługuje jedynie najprostszy typ kartridża, czyli ROM only. W przyszłości projekt wymaga rozszerzenia o obsługę innych typów kartridży wykorzystujących kontrolery banków pamięci (Memory Bank Controller). Implementacja tych mechanizmów umożliwiłaby uruchamianie znacznie większej liczby gier oraz zwiększyłaby funkcjonalność emulatora.

Należy również zauważyć, że w obecnej wersji projektu kod zawiera elementy zarówno stylu programowania charakterystycznego dla języka C, jak i dla języka C++. Takie podejście nie jest optymalnym rozwiązaniem z punktu widzenia spójności projektu i w przyszłości powinno zostać zastąpione bardziej jednolitą koncepcją implementacyjną. Refaktoryzacja kodu w kierunku konsekwentnego wykorzystania jednego stylu programowania mogłaby poprawić czytelność, łatwość utrzymania oraz dalszy rozwój projektu.

Podsumowując, realizacja emulatora Game Boy stanowiła wartościowe doświadczenie pozwalające na pogłębienie wiedzy z zakresu architektury komputerów, emulacji sprzętu oraz programowania niskopoziomowego. Jednocześnie projekt pozostawia przestrzeń do dalszego rozwoju, zarówno w zakresie rozszerzenia obsługi sprzętu, jak i poprawy jakości oraz struktury kodu.

Bibliografia

[1] Rodrigo Copetti. *Game Boy Architecture - A Practical Analysis*. Dostęp: 2025-02-15. URL: https://www.copetti.org/writings/consoles/game-boy/.

[2] *Game Boy*. Dostęp: 2025-02-16. Wikipedia. URL: https://en.wikipedia.org/wiki/Game_Boy.

[3] *Game Boy Opcode Tables*. Dostęp 2025-02-15. URL: https://gbdev.io/gb-opcodes/optables/.

[4] Gekkio. *Game Boy: Complete Technical Reference*. Dostęp: 2025-02-15. URL: https://gekkio.fi/files/gb-docs/gbctr.pdf.

[5] Shay Green. *Blargg's Game Boy Hardware Test ROMs*. Dostęp 2025-02-15. URL: https://gbdev.gg8.se/files/roms/blargg-gb-tests/.

[6] Lior Halphon. *Sameboy*. Dostęp: 2025-03-17. URL: https://sameboy.github.io.

[7] Robert Heaton. *Gameboy Doctor*. Dostęp 2025-02-15. URL: https://robertheaton.com/gameboy-doctor/.

[8] *ISO/IEC 14882:2020 – Programming Languages: C++*. Spraw. tech. International Organization for Standardization, 2020.

[9] *Pan Docs - Game Boy Technical Reference*. Dostęp 2025-02-15. URL: https://gbdev.io/pandocs/.

[10] Jeffrey Pfau. *mGBA – Game Boy Advance Emulator*. https://mgba.io. Dostęp: 18-03-2026. 2014.

[11] Ignacio Sanchez. *Gearboy – Game Boy / Game Boy Color Emulator*. https://github.com/drhelius/Gearboy. Dostęp: 18-03-2026. 2012.

[12] Sanqui. *2048-gb*. Dostęp: 2025-03-09. URL: https://github.com/Sanqui/2048-gb.

[13] Ramon Santamaria. *Raylib*. Dostęp: 2025-02-18. URL: https://www.raylib.com.

[14] Bjarne Stroustrup. *A Tour of C++*. 3 wyd. C++ In-Depth Series. Addison-Wesley, 2022. ISBN: 978-0-13-681648-5.

[15] VisualBoyAdvance-M Team. *VisualBoyAdvance-M – Game Boy Advance Emulator*. https://visualboyadvance-m.org. Dostęp: 18-03-2026. 2005.

[16] Victor Zverovich. *fmt*. Dostęp: 2025-02-25. URL: https://fmt.dev.